

Abstract Data Type

Abstract Data Type (ADT), ADT Pila

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



Abstract Data Type

Abstract Data Type



Definizione

In un dato linguaggio di programmazione, un Abstract Data Type (ADT) è costituito da:

1. La definizione di un tipo T
2. Un insieme di signature¹ di funzioni che operano su valori di tipo T
3. L'implementazione delle operazioni di cui al punto 2

tali che il codice cliente che si limita a usare T solo attraverso le operazioni del punto 2:

- È disaccoppiato dal codice di cui ai punti 1 e 3
 - (ovviamente dipende dal nome del tipo T)

¹Cioè il nome, tipo del valore di ritorno e tipo di ciascun parametro di una funzione

Abstract Data Type in C

- In C è possibile realizzare un ADT tramite:
 1. La definizione di un tipo T
 -
 2. Un insieme di signature¹ di funzioni che operano su valori di tipo T
 -
 3. L'implementazione delle operazioni di cui al punto 2
 -
-
-
-

Abstract Data Type in C

- In C è possibile realizzare un ADT tramite:
 1. La definizione di un tipo T
 - **Con una typedef**
 2. Un insieme di signature¹ di funzioni che operano su valori di tipo T
 -
 3. L'implementazione delle operazioni di cui al punto 2
 -
-
-
-

Abstract Data Type in C

- In C è possibile realizzare un ADT tramite:
 1. La definizione di un tipo T
 - Con una typedef
 2. Un insieme di signature¹ di funzioni che operano su valori di tipo T
 - Con un file .h che contiene la `typedef` (del punto 1) e i prototipi delle funzioni (del punto 3)
 3. L'implementazione delle operazioni di cui al punto 2
 -
-
-
-

Abstract Data Type in C

- In C è possibile realizzare un ADT tramite:
 1. La definizione di un tipo T
 - Con una typedef
 2. Un insieme di signature¹ di funzioni che operano su valori di tipo T
 - Con un file .h che contiene la `typedef` (del punto 1) e i prototipi delle funzioni (del punto 3)
 3. L'implementazione delle operazioni di cui al punto 2
 - Con un file .c che contiene l'implementazione delle funzioni che realizzano le operazioni

•

•

•

Abstract Data Type in C

- In C è possibile realizzare un ADT tramite:
 1. La definizione di un tipo T
 - Con una typedef
 2. Un insieme di signature¹ di funzioni che operano su valori di tipo T
 - Con un file .h che contiene la `typedef` (del punto 1) e i prototipi delle funzioni (del punto 3)
 3. L'implementazione delle operazioni di cui al punto 2
 - Con un file .c che contiene l'implementazione delle funzioni che realizzano le operazioni
- Abbiamo già visto degli esempi di ADT?
 -
 -

Abstract Data Type in C

- In C è possibile realizzare un ADT tramite:
 1. La definizione di un tipo T
 - Con una typedef
 2. Un insieme di signature¹ di funzioni che operano su valori di tipo T
 - Con un file .h che contiene la `typedef` (del punto 1) e i prototipi delle funzioni (del punto 3)
 3. L'implementazione delle operazioni di cui al punto 2
 - Con un file .c che contiene l'implementazione delle funzioni che realizzano le operazioni
- Abbiamo già visto degli esempi di ADT?
 - Sì, quando abbiamo definito le funzioni per i numeri complessi come struct opaca
 -

Abstract Data Type in C

- In C è possibile realizzare un ADT tramite:
 1. La definizione di un tipo T
 - Con una typedef
 2. Un insieme di signature¹ di funzioni che operano su valori di tipo T
 - Con un file .h che contiene la `typedef` (del punto 1) e i prototipi delle funzioni (del punto 3)
 3. L'implementazione delle operazioni di cui al punto 2
 - Con un file .c che contiene l'implementazione delle funzioni che realizzano le operazioni
- Abbiamo già visto degli esempi di ADT?
 - Sì, quando abbiamo definito le funzioni per i numeri complessi come struct opaca
 - In modo meno esplicito, con le liste

ADT Fondamentali

- Pila
 - In inglese: stack
- Coda
 - In inglese: queue
- Insieme
 - In inglese: set

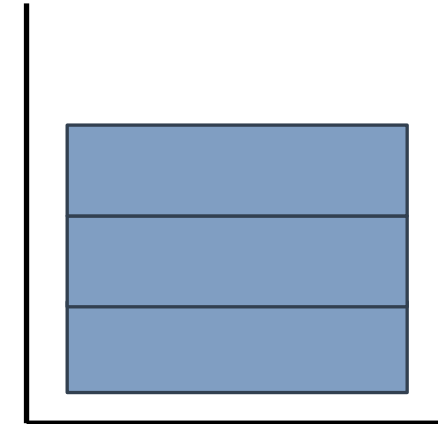




Pila (Stack)

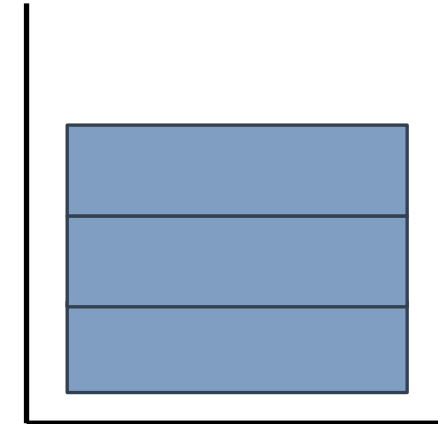
Pila (Stack)

- Una pila (stack) è:
 - una sovrapposizione di elementi dello stesso tipo
-
-
-
-



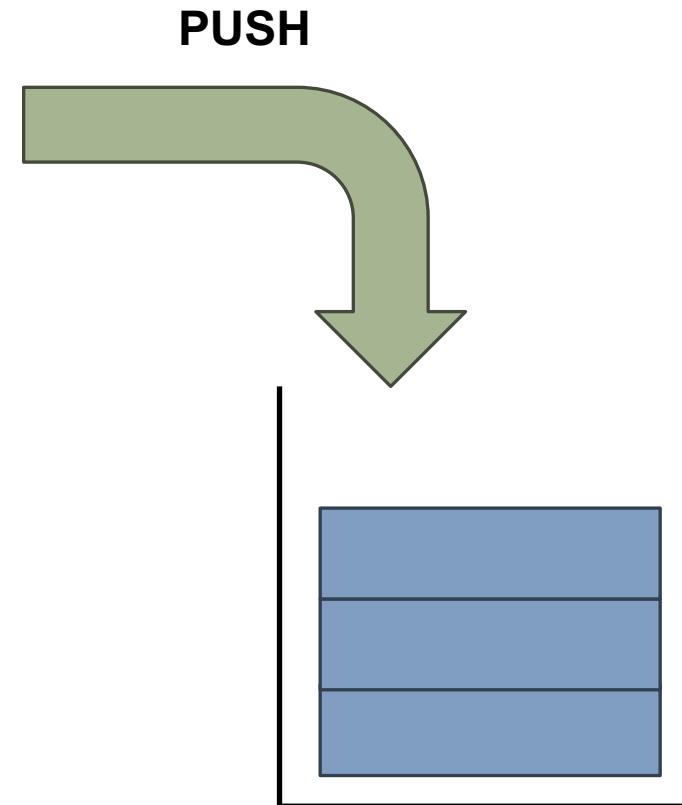
Pila (Stack)

- Una pila (stack) è:
 - una sovrapposizione di elementi dello stesso tipo
- Una pila è gestita
 - Seguendo la politica LIFO (Last-In-First-Out)
 -
 -



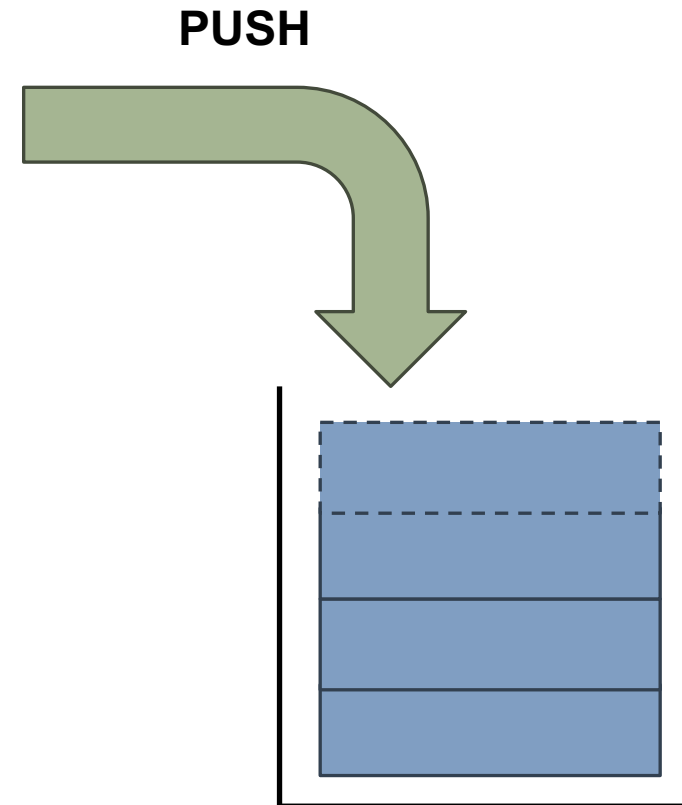
Pila (Stack)

- Una pila (stack) è:
 - una sovrapposizione di elementi dello stesso tipo
- Una pila è gestita
 - Seguendo la politica LIFO (Last-In-First-Out)
 - Aggiungo in cima: **PUSH**
 -



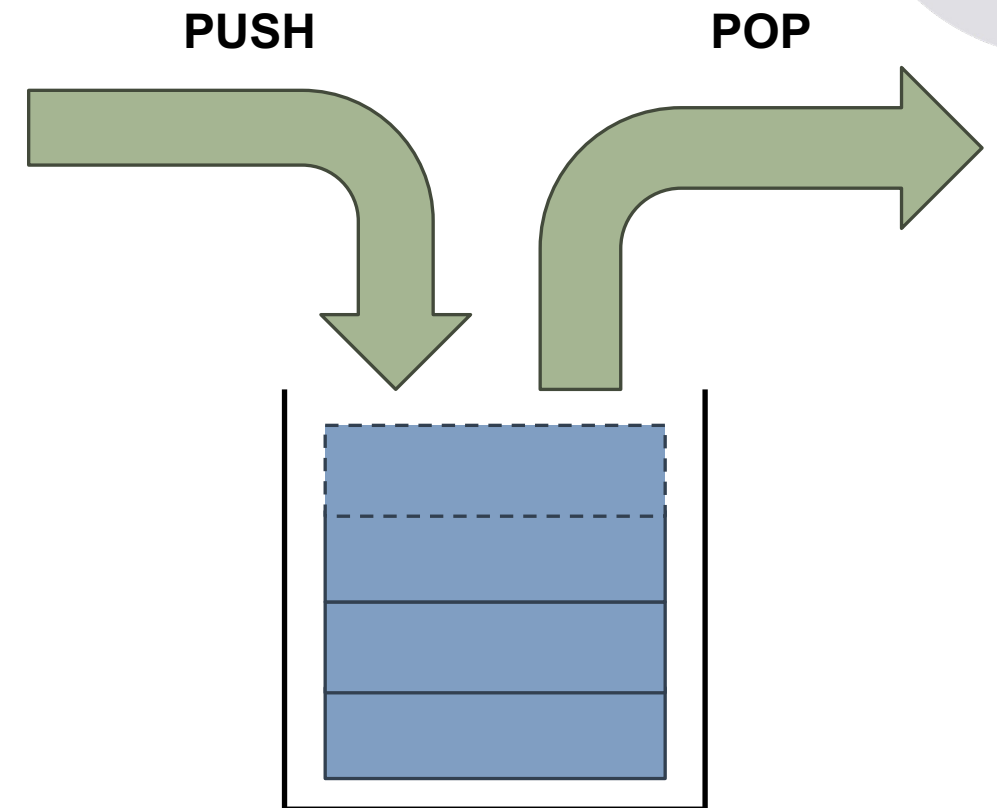
Pila (Stack)

- Una pila (stack) è:
 - una sovrapposizione di elementi dello stesso tipo
- Una pila è gestita
 - Seguendo la politica LIFO (Last-In-First-Out)
 - Aggiungo in cima: **PUSH**
 -



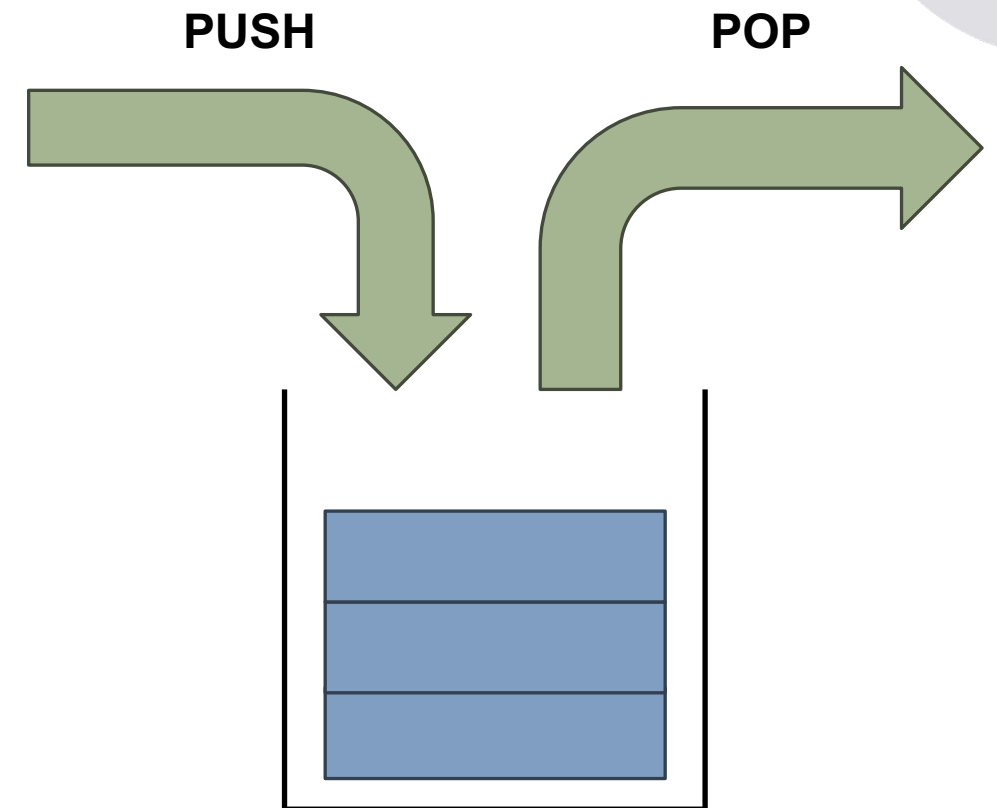
Pila (Stack)

- Una pila (stack) è:
 - una sovrapposizione di elementi dello stesso tipo
- Una pila è gestita
 - Seguendo la politica LIFO (Last-In-First-Out)
 - Aggiungo in cima: **PUSH**
 - Tolgo in cima (ultimo inserito): **POP**

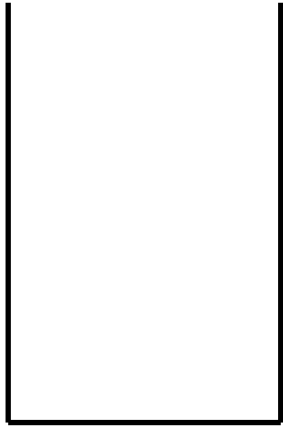


Pila (Stack)

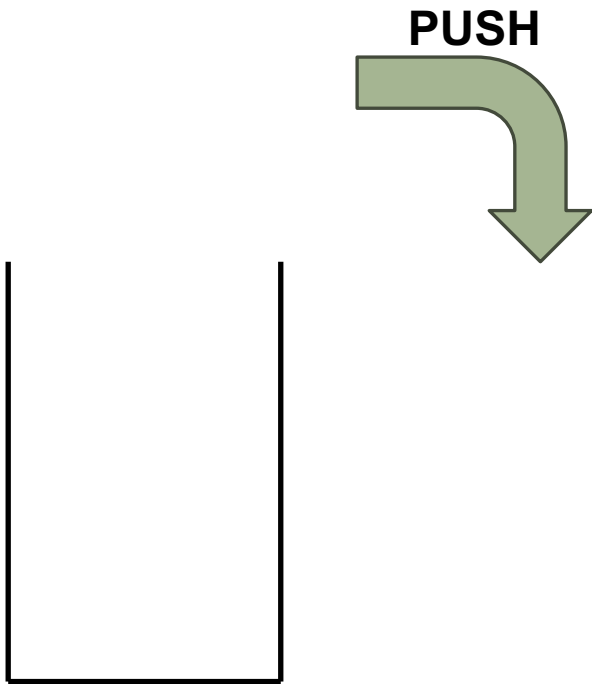
- Una pila (stack) è:
 - una sovrapposizione di elementi dello stesso tipo
- Una pila è gestita
 - Seguendo la politica LIFO (Last-In-First-Out)
 - Aggiungo in cima: **PUSH**
 - Tolgo in cima (ultimo inserito): **POP**



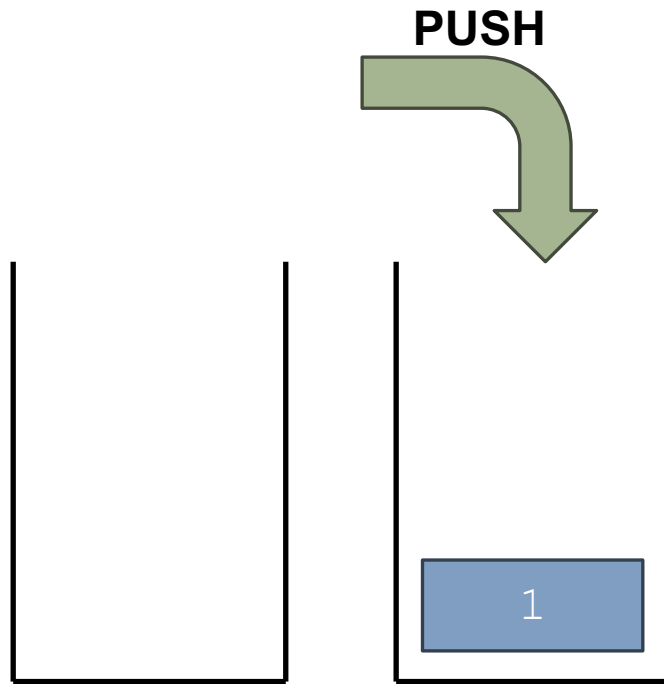
Pila (Stack): Esempio



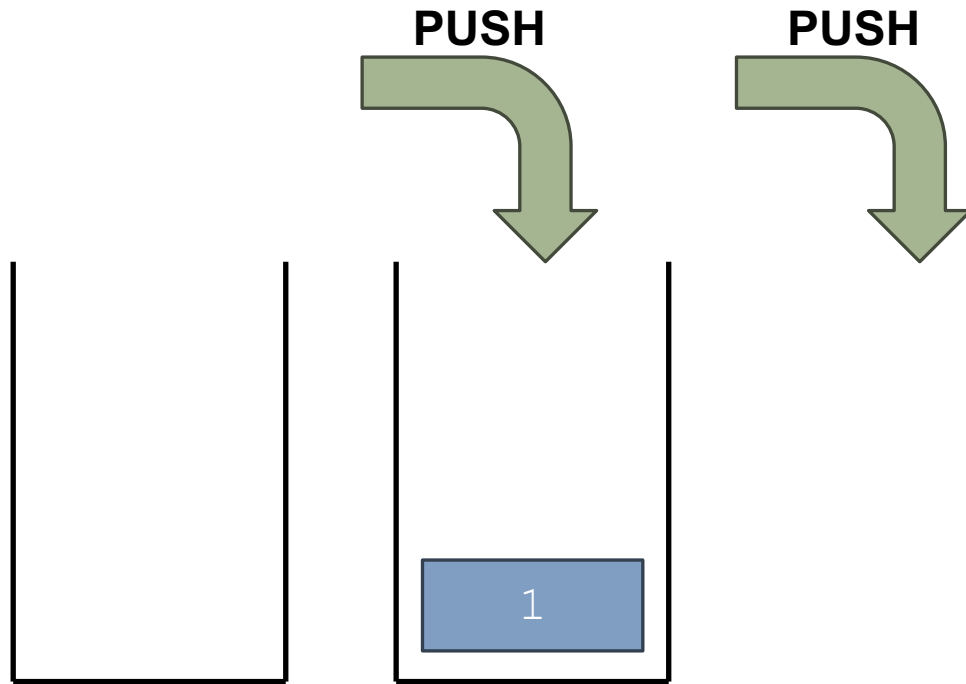
Pila (Stack): Esempio



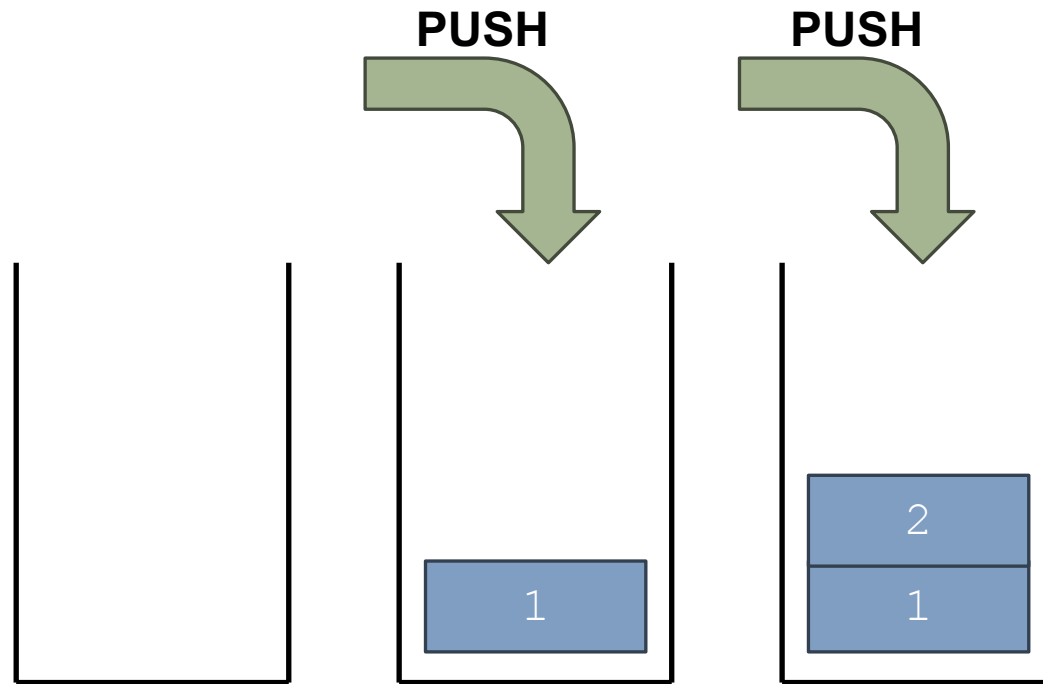
Pila (Stack): Esempio



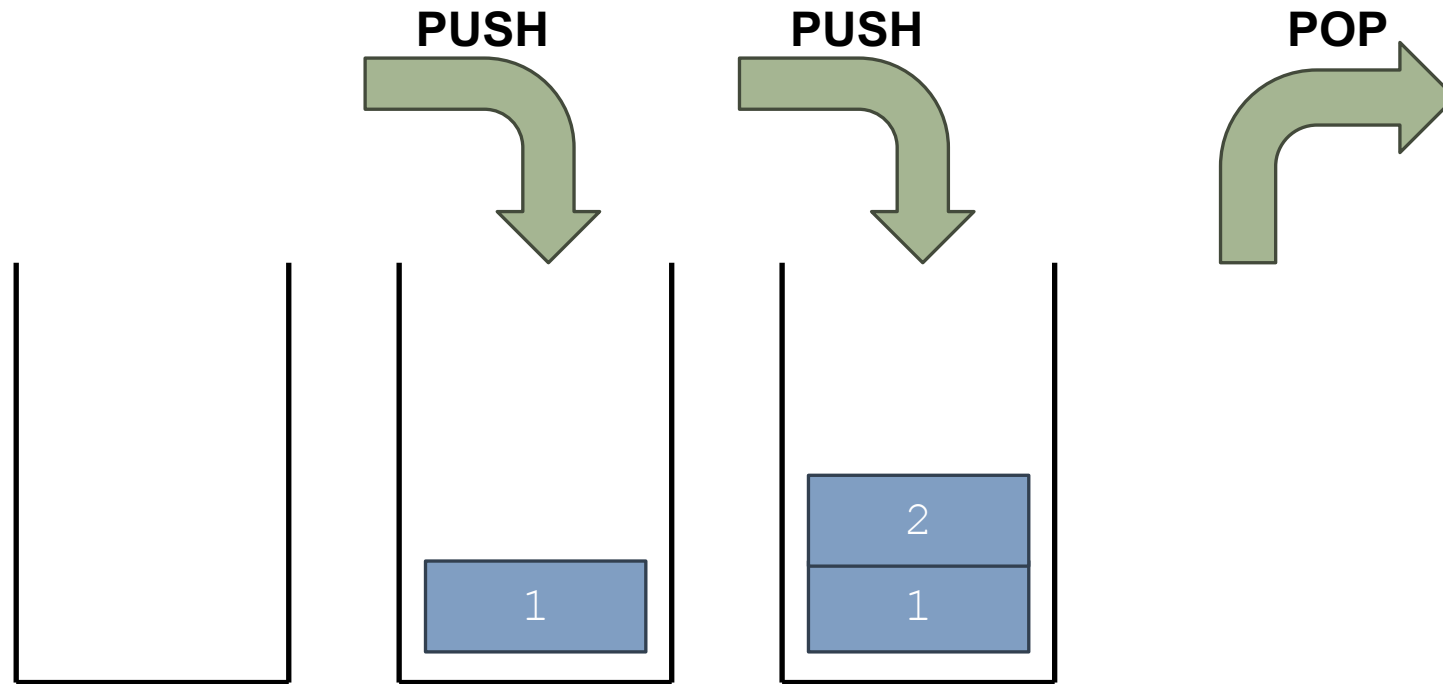
Pila (Stack): Esempio



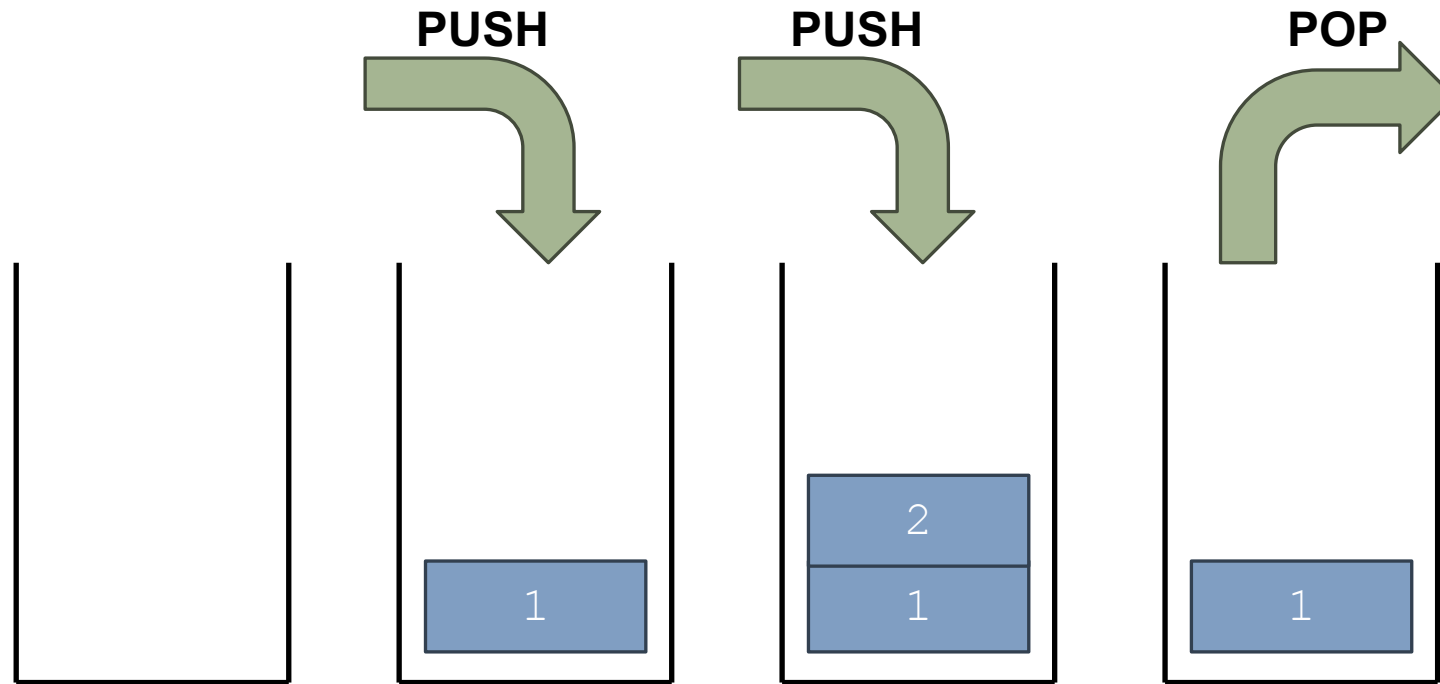
Pila (Stack): Esempio



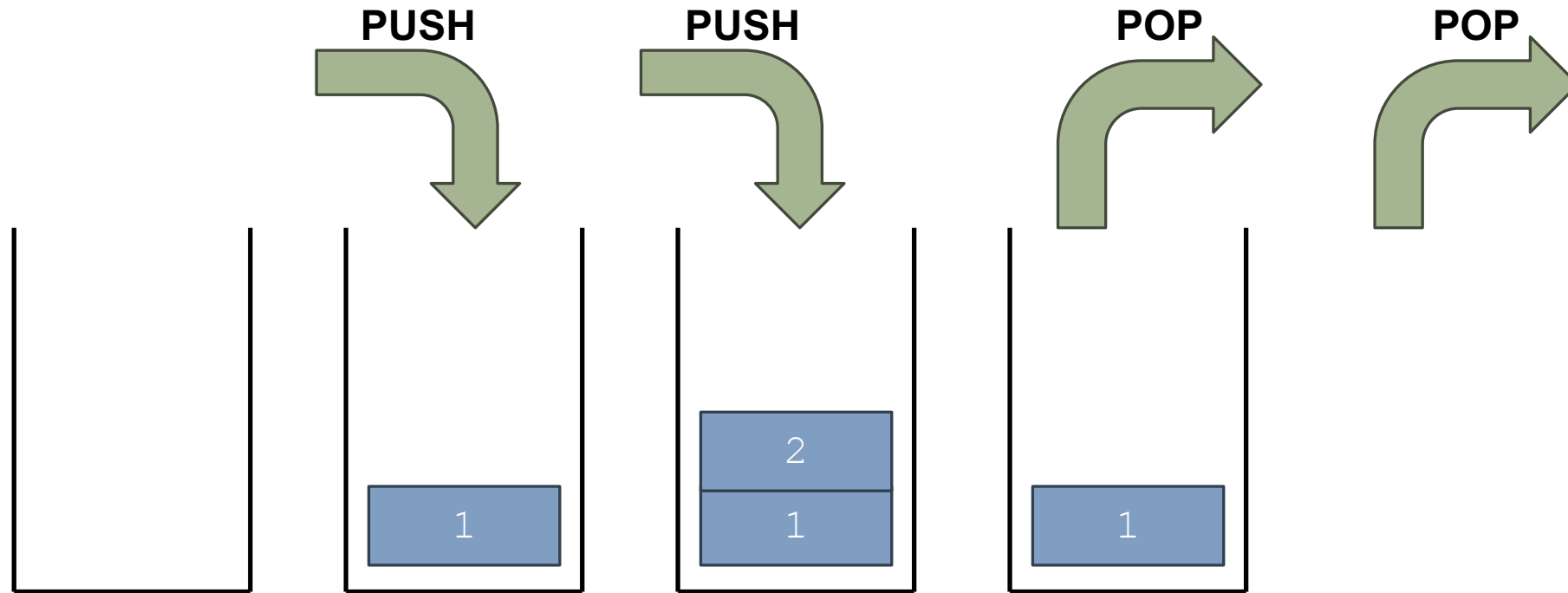
Pila (Stack): Esempio



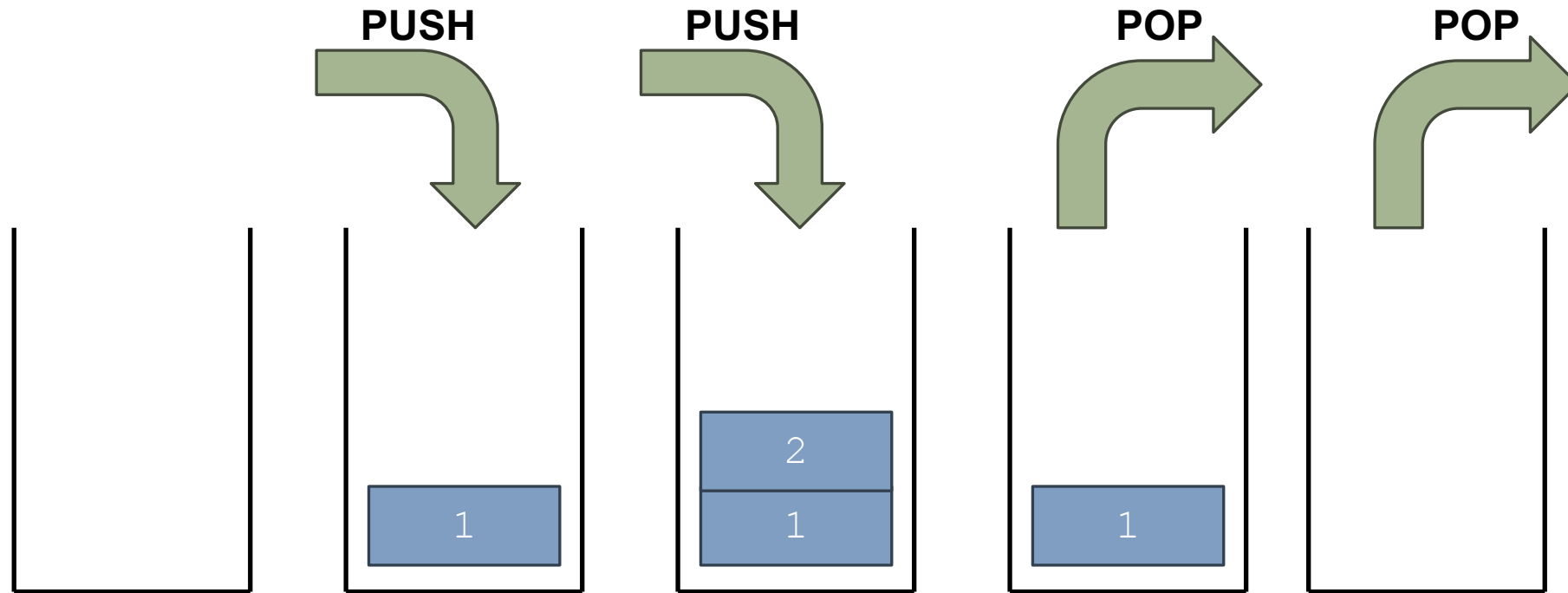
Pila (Stack): Esempio



Pila (Stack): Esempio



Pila (Stack): Esempio



Stack (Pila) in C I



1.

•

2.

•

•

•

•

•

•

•

•

Stack (Pila) in C I

1. Nome del tipo

- Stack (di Element)

2.

- -
- -
- -
- -
- -
- -

Stack (Pila) in C I

1. Nome del tipo

- `Stack (di Element)`

2. Signature e semantica (astratta) delle operazioni fondamentali:

- `Stack mkStack()`
 - Restituisce una pila vuota
- `void push(Stack, const Element)`
 - Aggiunge un elemento in cima alla pila
- `Element pop(Stack)`
 - Toglie e restituisce l'elemento in cima alla pila
- `_Bool isEmptyStack(const Stack)`
 - Controlla se la pila è vuota

Stack (Pila) in C II



2. ... altre operazioni, non sempre presenti sono:

- `Element top(const Stack)`
 - Restituisce l'elemento in cima alla pila (senza toglierlo)
- `int stackSize(const Stack)`
 - Restituisce il numero degli elementi presenti nella pila
- `_Bool isFullStack(const Stack)`
 - Controlla se la pila è piena
- `void dsStack(Stack*)`
 - Distrugge la pila, recuperando la memoria

Stack (Pila) in C III

2. ... varianti di `mkStack`, `push` e `pop`:

- `Stack mkStack(int)`
 - Crea una pila vuota di capacità (fissa o variabile) specificata
- `_Bool push(Stack, const Element)`
 - Aggiunge un elemento in cima alla pila e restituisce l'esito dell'operazione
- `_Bool pop(Stack)`
 - Toglie l'elemento in cima alla pila e restituisce l'esito dell'operazione

3.

-
-

Stack (Pila) in C III

2. ... varianti di `mkStack`, `push` e `pop`:

- `Stack mkStack(int)`
 - Crea una pila vuota di capacità (fissa o variabile) specificata
- `_Bool push(Stack, const Element)`
 - Aggiunge un elemento in cima alla pila e restituisce l'esito dell'operazione
- `_Bool pop(Stack)`
 - Toglie l'elemento in cima alla pila e restituisce l'esito dell'operazione

Perché
possono fallire?

3.

-
-

Stack (Pila) in C III

2. ... varianti di `mkStack`, `push` e `pop`:

- `Stack mkStack(int)`
 - Crea una pila vuota di capacità (fissa o variabile) specificata
- `_Bool push(Stack, const Element)`
 - Aggiunge un elemento in cima alla pila e restituisce l'esito dell'operazione
- `_Bool pop(Stack)`
 - Toglie l'elemento in cima alla pila e restituisce l'esito dell'operazione

Perché
possono fallire?

3. Implementazioni, ne vedremo due:

-
-

Stack (Pila) in C III

2. ... varianti di `mkStack`, `push` e `pop`:

- `Stack mkStack(int)`
 - Crea una pila vuota di capacità (fissa o variabile) specificata
- `_Bool push(Stack, const Element)`
 - Aggiunge un elemento in cima alla pila e restituisce l'esito dell'operazione
- `_Bool pop(Stack)`
 - Toglie l'elemento in cima alla pila e restituisce l'esito dell'operazione

Perché
possono fallire?

3. Implementazioni, ne vedremo due:

- Tramite lista
-

Stack (Pila) in C III

2. ... varianti di `mkStack`, `push` e `pop`:

- `Stack mkStack(int)`
 - Crea una pila vuota di capacità (fissa o variabile) specificata
- `_Bool push(Stack, const Element)`
 - Aggiunge un elemento in cima alla pila e restituisce l'esito dell'operazione
- `_Bool pop(Stack)`
 - Toglie l'elemento in cima alla pila e restituisce l'esito dell'operazione

Perché
possono fallire?

3. Implementazioni, ne vedremo due:

- Tramite lista
- Tramite array

Esempio: Pila di char

- Nome del tipo:
 - `CharStackADT`
- Consideriamo sei operazioni:
 - `CharStackADT mkStack()`
 - Restituisce una pila vuota
 - `void dsStack(CharStackADT *ps)`
 - Distrugge la pila
 - `Bool push(CharStackADT s, char e)`
 - Aggiunge un `char` alla pila
 - `char pop(CharStackADT s)`
 - Toglie e restituisce il `char` in cima alla pila
 - `_Bool isEmptyStack(const CharStackADT s)`
 - Controlla se la pila è vuota
 - `int stackSize(const CharStackADT s)`
 - Restituisce il numero di elementi nella pila
- Tipo e signature definite in `charStackADT.h`

test_charStackADT.c



- Come procediamo?

-

-

-

test_charStackADT.c

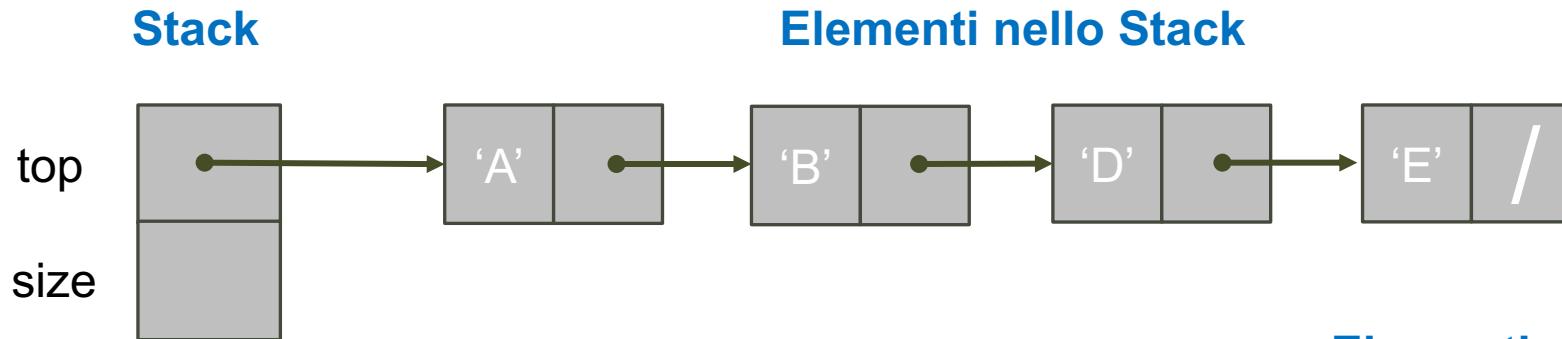


- Come procediamo?
- In modo iterativo e incrementale applicando la metodologia TDD
 -
 -

test_charStackADT.c

- Come procediamo?
- In modo iterativo e incrementale applicando la metodologia TDD
 - Sviluppiamo (due) implementazioni per la specifica `charStackADT.h` partendo da una test suite `test_charStackADT.c`
 - Test suite omessa nei lucidi e lasciata come esercizio a voi

Pila di char con Liste



- Implementazione con liste linkate:
 - La lista può crescere senza limiti
- Operazioni push/pop
 - Inserzione/estrazione dalla testa (top)
- L'elemento in coda
 - è il primo a essere stato inserito
 - Il più "profondo" nello stack

Elementi nello Stack, i.e., Lista

```
typedef struct listNode ListNode,  
        *ListNodePtr;  
  
struct listNode {  
    char data;  
    ListNodePtr next;  
};
```

Stack

```
struct charStack {  
    ListNodePtr top;  
    int size;  
};
```

Pila di char con Liste: mkStack ()

- Crea uno stack
 - Con lista vuota
 - E 0 elementi

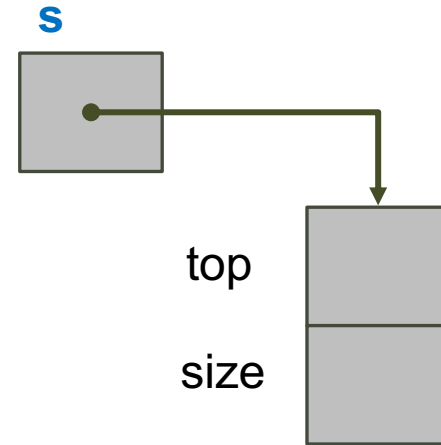
```
CharStackADT mkStack () {
```

```
}
```

Pila di char con Liste: mkStack ()

- Crea uno stack
 - Con lista vuota
 - E 0 elementi

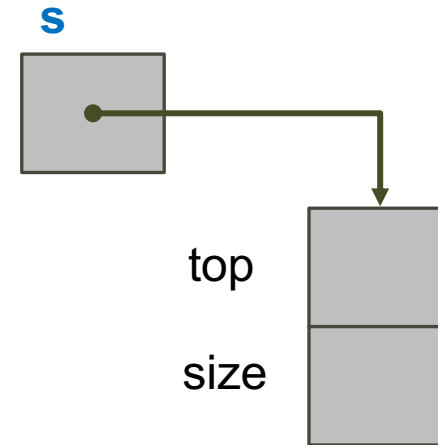
```
CharStackADT mkStack() {  
    CharStackADT s = malloc(sizeof(*s));
```



Pila di char con Liste: mkStack ()

- Crea uno stack
 - Con lista vuota
 - E 0 elementi

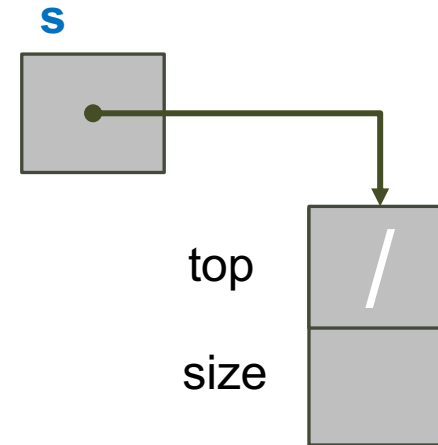
```
CharStackADT mkStack() {  
    CharStackADT s = malloc(sizeof(*s));  
  
    // malloc() fallita?  
    if (s == NULL)  
        return NULL;  
  
}
```



Pila di char con Liste: mkStack ()

- Crea uno stack
 - Con lista vuota
 - E 0 elementi

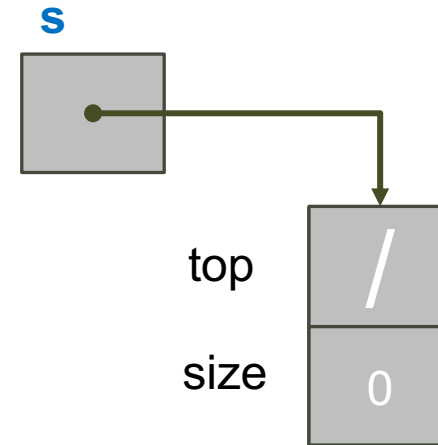
```
CharStackADT mkStack() {  
    CharStackADT s = malloc(sizeof(*s));  
  
    // malloc() fallita?  
    if (s == NULL)  
        return NULL;  
  
    s->top = NULL; // lista vuota  
  
}
```



Pila di char con Liste: mkStack ()

- Crea uno stack
 - Con lista vuota
 - E 0 elementi

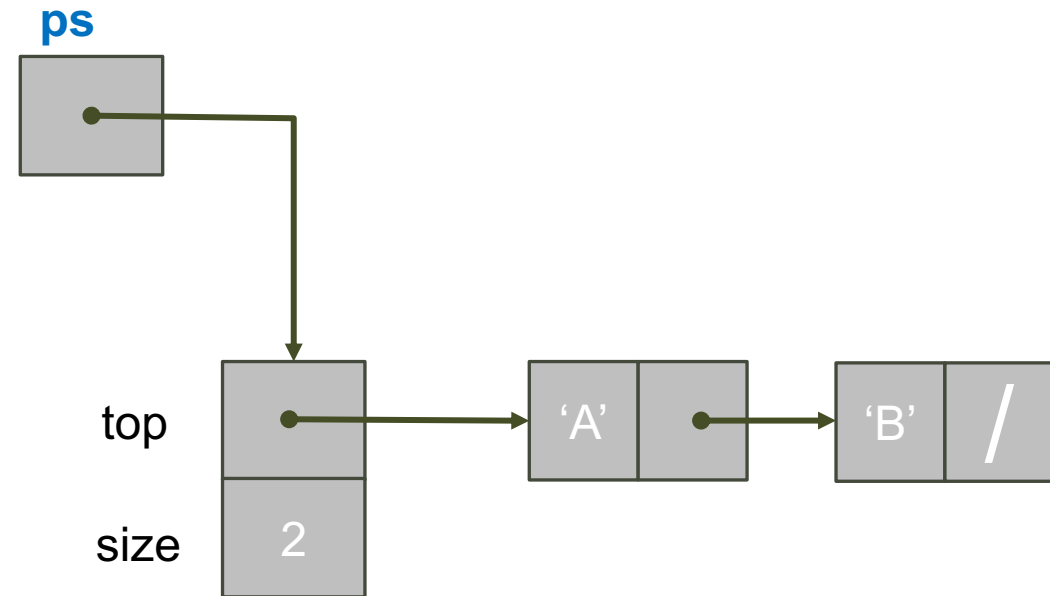
```
CharStackADT mkStack() {  
    CharStackADT s = malloc(sizeof(*s));  
  
    // malloc() fallita?  
    if (s == NULL)  
        return NULL;  
  
    s->top = NULL; // lista vuota  
    s->size = 0;   // con 0 elementi  
    return s;  
}
```



Pila di char con Liste: dsStack()

- Distrugge uno stack
 - Libera la memoria sia dello stack che di tutti i suoi elementi

```
void dsStack(CharStackADT *ps) {  
  
  
  
  
  
  
  
  
  
}
```



Pila di char con Liste: dsStack ()

- Distrugge uno stack
 - Libera la memoria sia dello stack che di tutti i suoi elementi

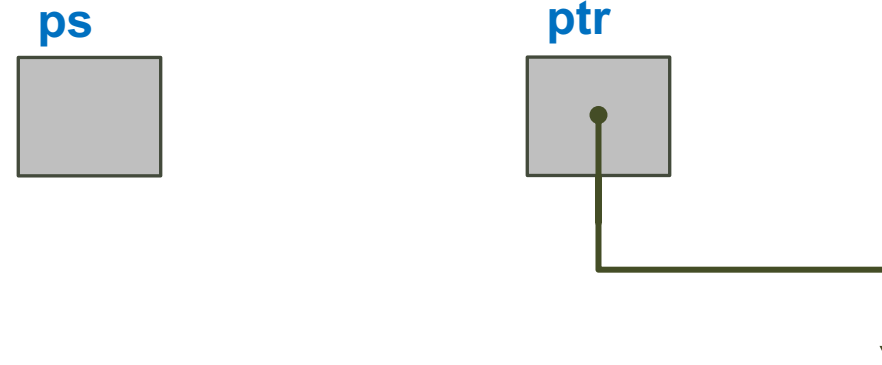
```
void dsStack(CharStackADT *ps) {  
    // Libera memoria degli elementi  
    ListNodePtr ptr = (*ps)->top;  
  
}
```



Pila di char con Liste: dsStack ()

- Distrugge uno stack
 - Libera la memoria sia dello stack che di tutti i suoi elementi

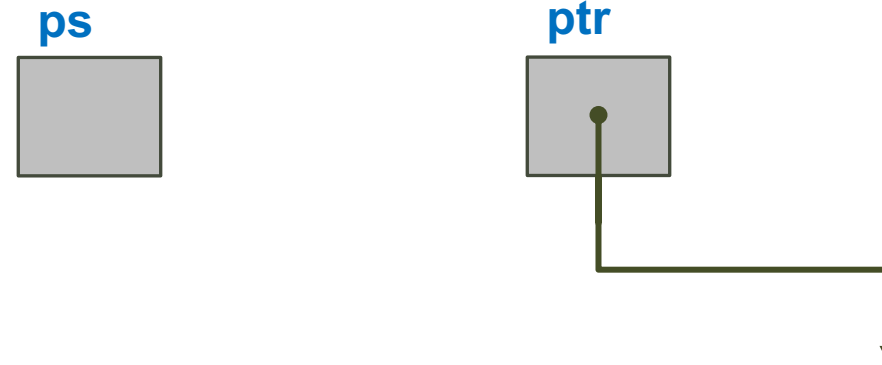
```
void dsStack(CharStackADT *ps) {  
    // Libera memoria degli elementi  
    ListNodePtr ptr = (*ps)->top;  
    while (ptr != NULL) {  
        ListNodePtr tmp = ptr;  
        ptr = ptr->next;  
    }  
}
```



Pila di char con Liste: dsStack()

- Distrugge uno stack
 - Libera la memoria sia dello stack che di tutti i suoi elementi

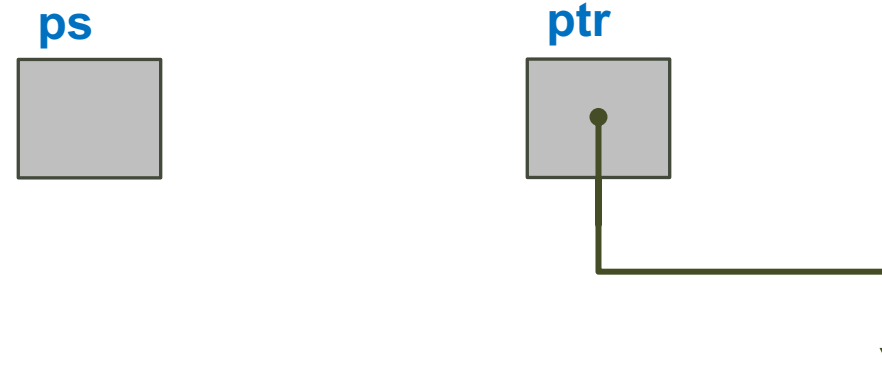
```
void dsStack(CharStackADT *ps) {  
    // Libera memoria degli elementi  
    ListNodePtr ptr = (*ps)->top;  
    while (ptr != NULL) {  
        ListNodePtr tmp = ptr;  
        ptr = ptr->next;  
        free(tmp);  
    }  
}
```



Pila di char con Liste: dsStack ()

- Distrugge uno stack
 - Libera la memoria sia dello stack che di tutti i suoi elementi

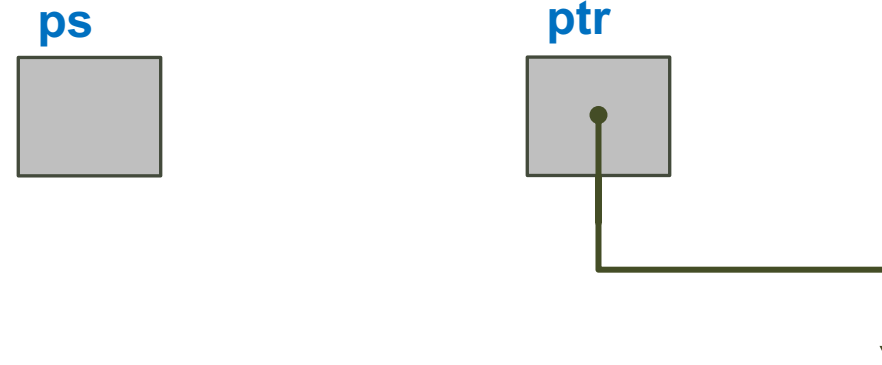
```
void dsStack(CharStackADT *ps) {  
    // Libera memoria degli elementi  
    ListNodePtr ptr = (*ps)->top;  
    while (ptr != NULL) {  
        ListNodePtr tmp = ptr;  
        ptr = ptr->next;  
        free(tmp);  
    }  
}
```



Pila di char con Liste: dsStack ()

- Distrugge uno stack
 - Libera la memoria sia dello stack che di tutti i suoi elementi

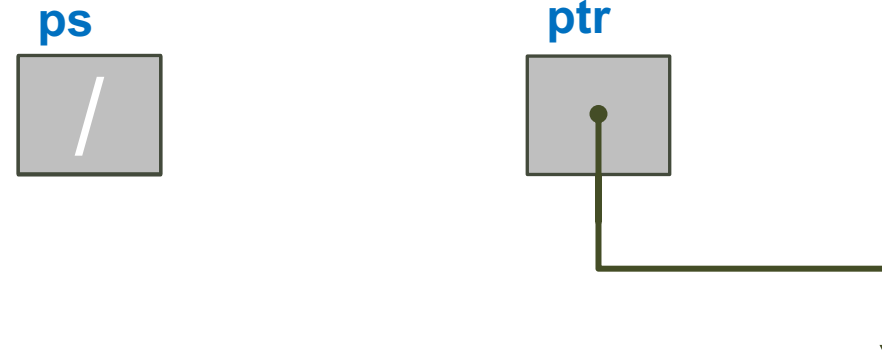
```
void dsStack(CharStackADT *ps) {  
    // Libera memoria degli elementi  
    ListNodePtr ptr = (*ps)->top;  
    while (ptr != NULL) {  
        ListNodePtr tmp = ptr;  
        ptr = ptr->next;  
        free(tmp);  
    }  
    // Libera memoria stack  
    free(*ps);  
}
```



Pila di char con Liste: dsStack ()

- Distrugge uno stack
 - Libera la memoria sia dello stack che di tutti i suoi elementi

```
void dsStack(CharStackADT *ps) {  
    // Libera memoria degli elementi  
    ListNodePtr ptr = (*ps)->top;  
    while (ptr != NULL) {  
        ListNodePtr tmp = ptr;  
        ptr = ptr->next;  
        free(tmp);  
    }  
    // Libera memoria stack  
    free(*ps);  
    *ps = NULL; // e lo marca NULL  
}
```

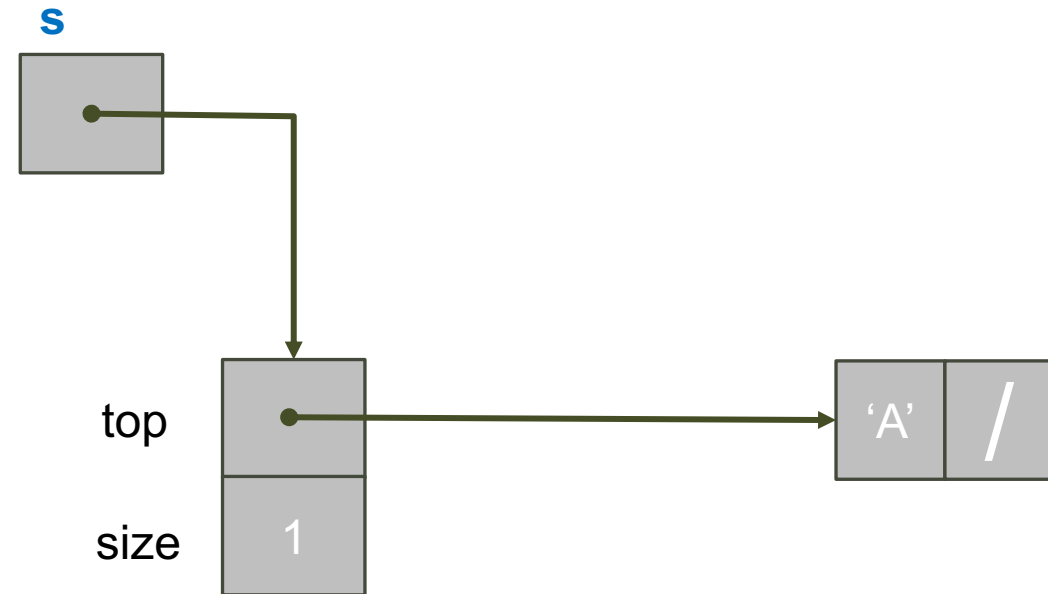


Pila di char con Liste: push ()

- Aggiunge un elemento
 - In testa alla lista

```
_Bool push(CharStackADT s, char e) {
```

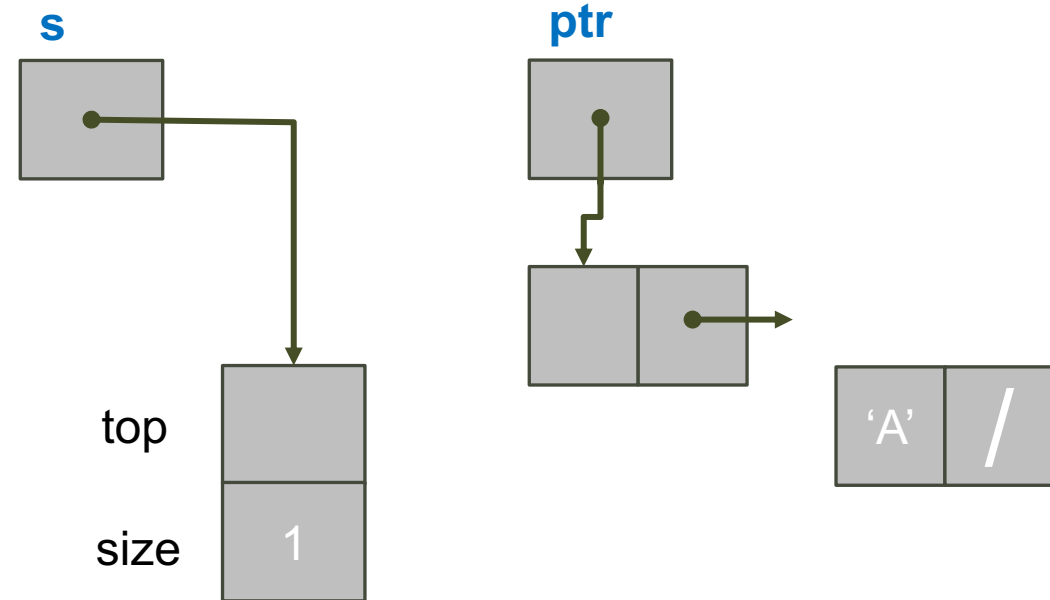
```
}
```



Pila di char con Liste: push ()

- Aggiunge un elemento
 - In testa alla lista

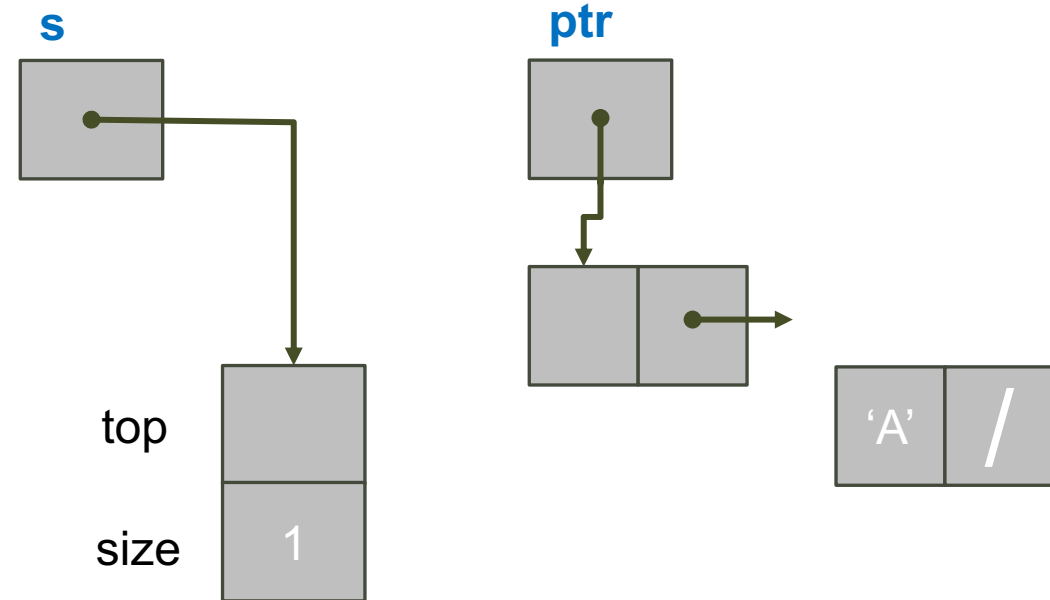
```
_Bool push(CharStackADT s, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
  
  
  
  
  
  
  
  
  
}
```



Pila di char con Liste: push ()

- Aggiunge un elemento
 - In testa alla lista

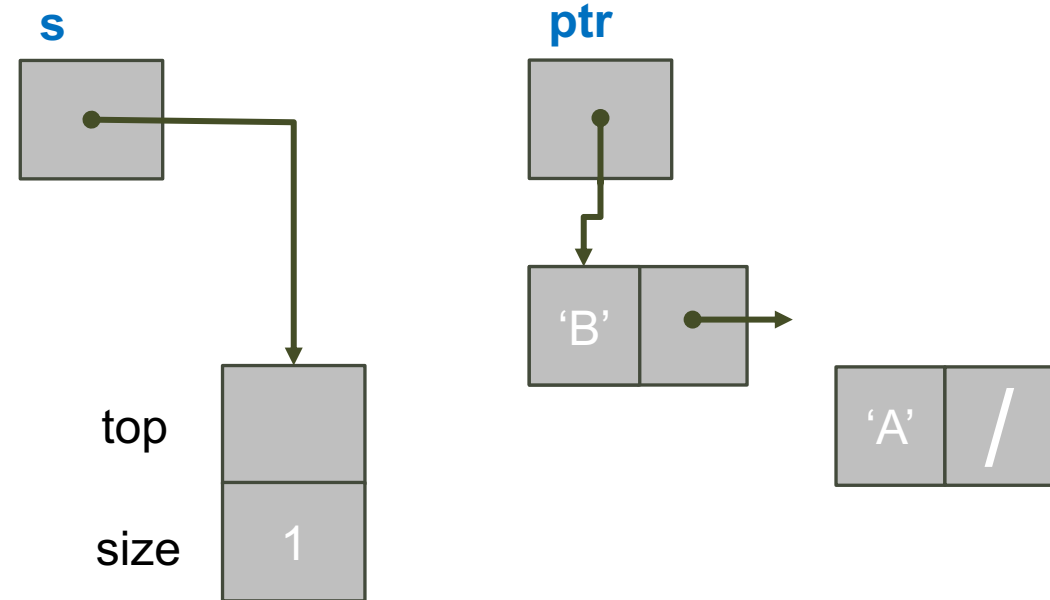
```
_Bool push(CharStackADT s, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL)  
        return 0;  
    return 1;  
}
```



Pila di char con Liste: push ()

- Aggiunge un elemento
 - In testa alla lista

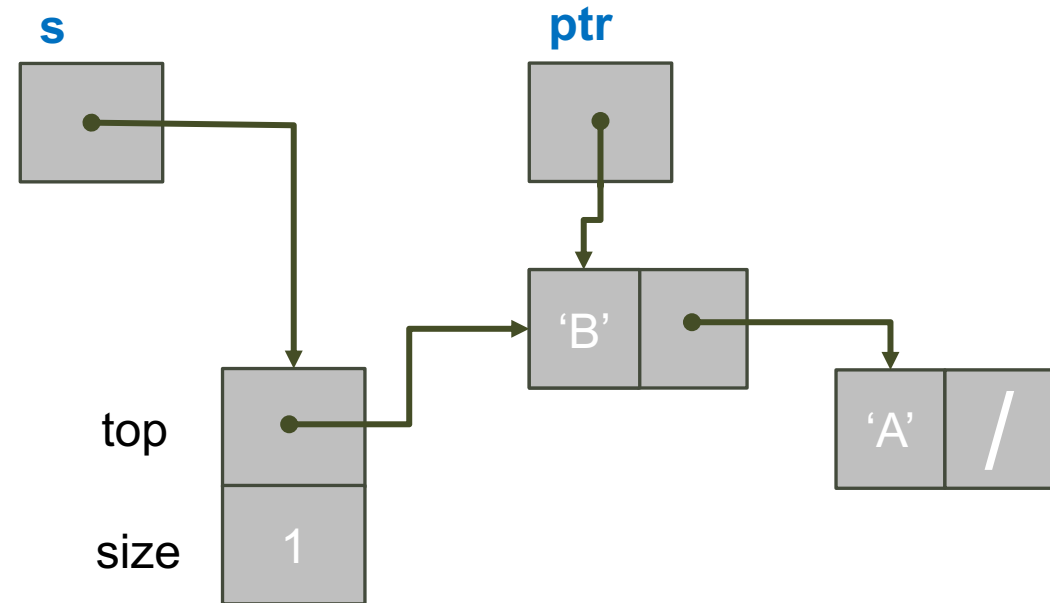
```
_Bool push(CharStackADT s, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL)  
        return 0;  
    // Salva dato  
    ptr->data = e;  
}
```



Pila di char con Liste: push ()

- Aggiunge un elemento
 - In testa alla lista

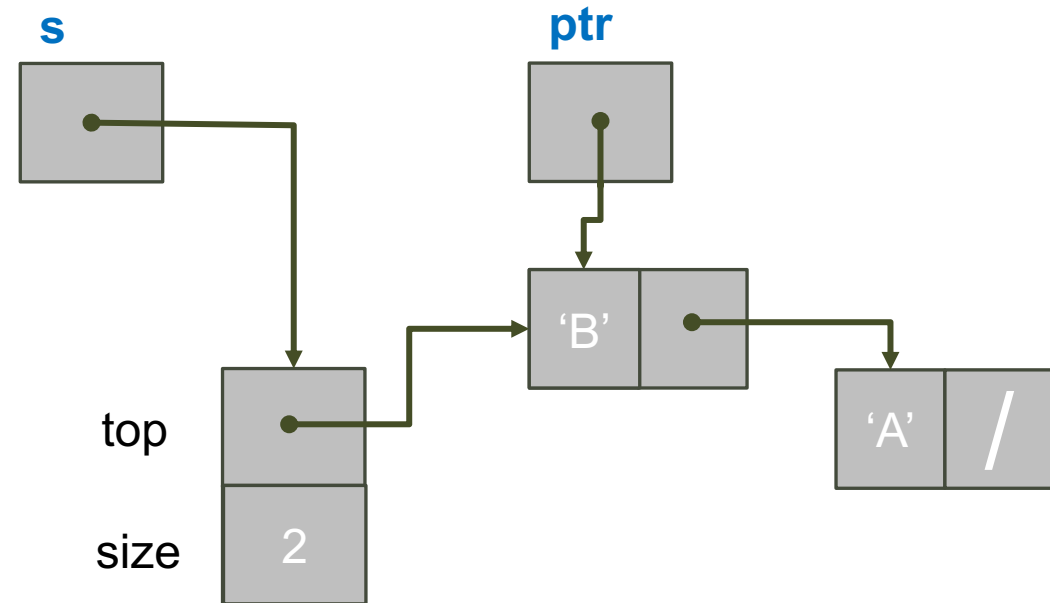
```
Bool push(CharStackADT s, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL)  
        return 0;  
    // Salva dato  
    ptr->data = e;  
    // Inserisci in testa  
    ptr->next = s->top;  
    s->top = ptr;  
}
```



Pila di char con Liste: push ()

- Aggiunge un elemento
 - In testa alla lista

```
Bool push(CharStackADT s, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL)  
        return 0;  
    // Salva dato  
    ptr->data = e;  
    // Inserisci in testa  
    ptr->next = s->top;  
    s->top = ptr;  
    // Incrementa dimensione stack  
    s->size++;  
    return 1;  
}
```

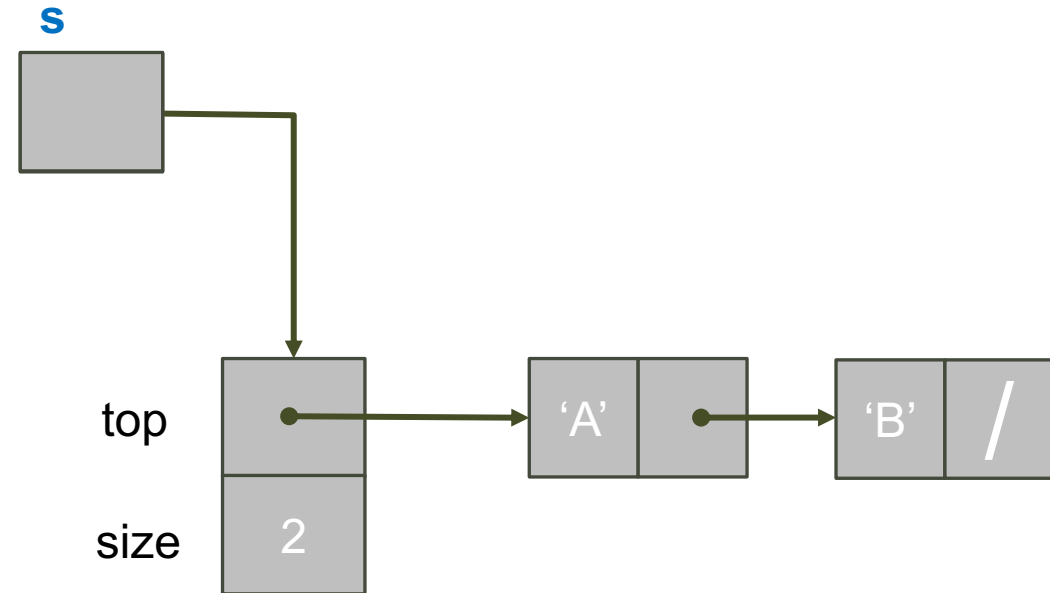


Pila di char con Liste: pop ()

- Toglie e restituisce l'elemento in cima alla pila

```
char pop(CharStackADT s) {
```

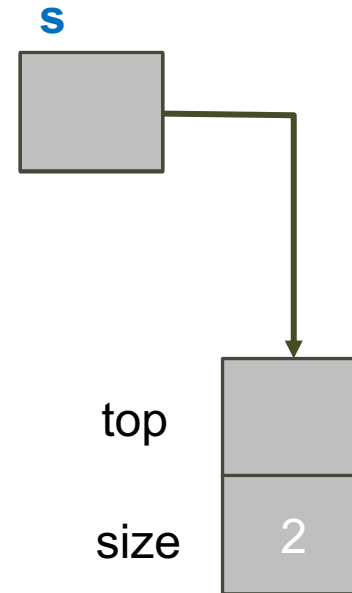
```
}
```



Pila di char con Liste: pop ()

- Toglie e restituisce l'elemento in cima alla pila

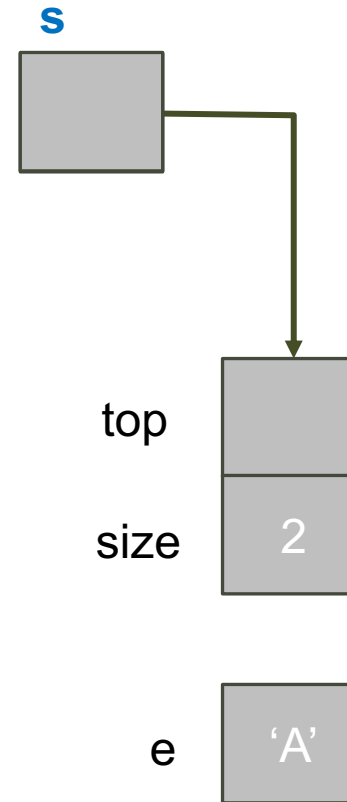
```
char pop(CharStackADT s) {  
    // Lista vuota?  
    if(isEmptyStack(s)) {  
        exit(EXIT_FAILURE);  
    }  
  
}
```



Pila di char con Liste: pop ()

- Toglie e restituisce l'elemento in cima alla pila

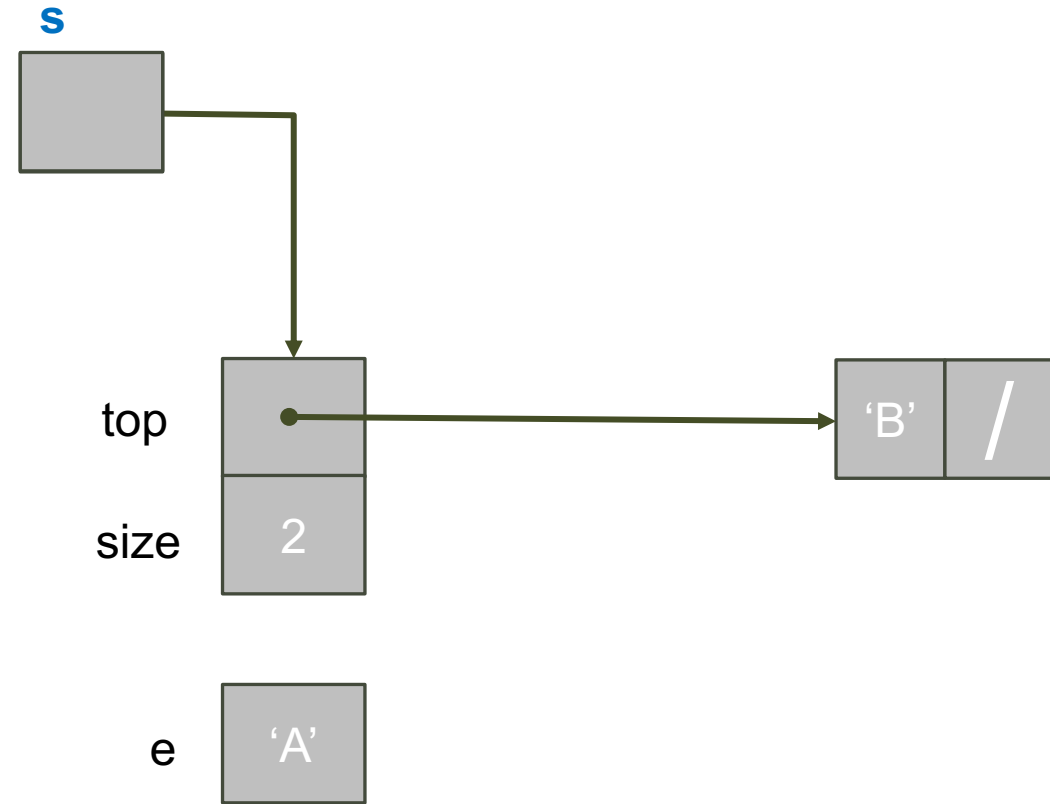
```
char pop(CharStackADT s) {  
    // Lista vuota?  
    if(isEmptyStack(s)){  
        exit(EXIT_FAILURE);  
    }  
    // Salvo dato in testa  
    char e = s->top->data;  
  
}
```



Pila di char con Liste: pop ()

- Toglie e restituisce l'elemento in cima alla pila

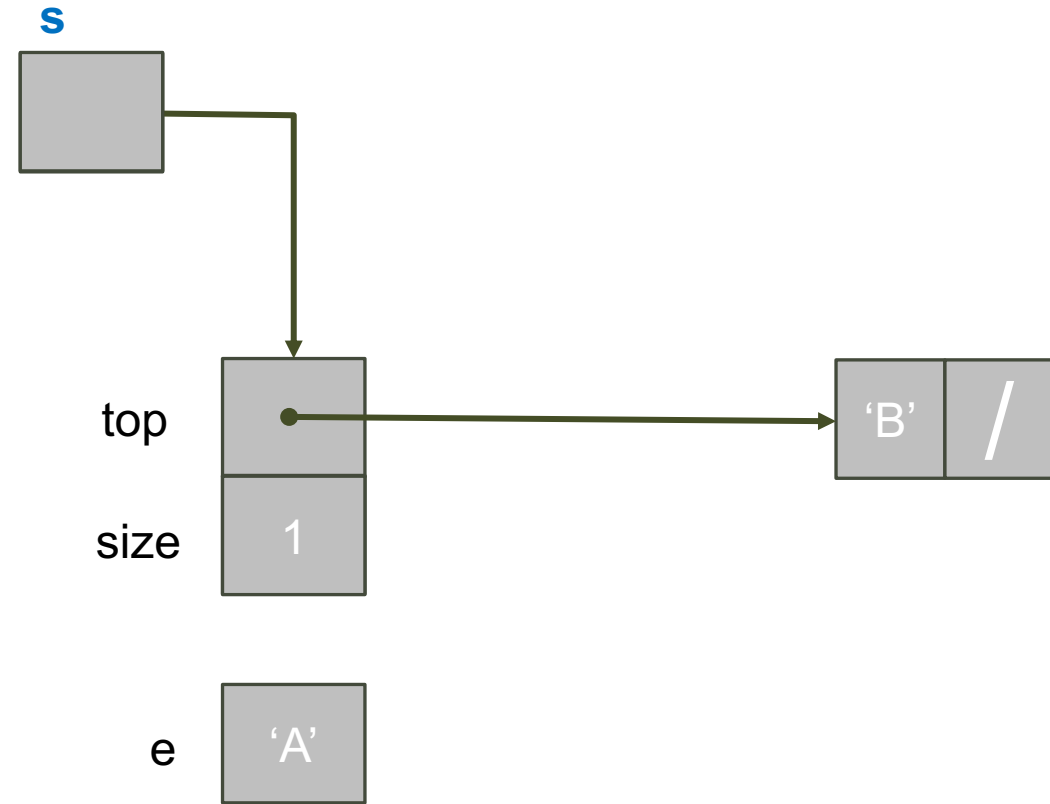
```
char pop(CharStackADT s) {  
    // Lista vuota?  
    if(isEmptyStack(s)){  
        exit(EXIT_FAILURE);  
    }  
    // Salvo dato in testa  
    char e = s->top->data;  
    // Cancella nodo di testa  
    ListNodePtr ptr = s->top;  
    s->top = ptr->next;  
    free(ptr);  
}
```



Pila di char con Liste: pop ()

- Toglie e restituisce l'elemento in cima alla pila

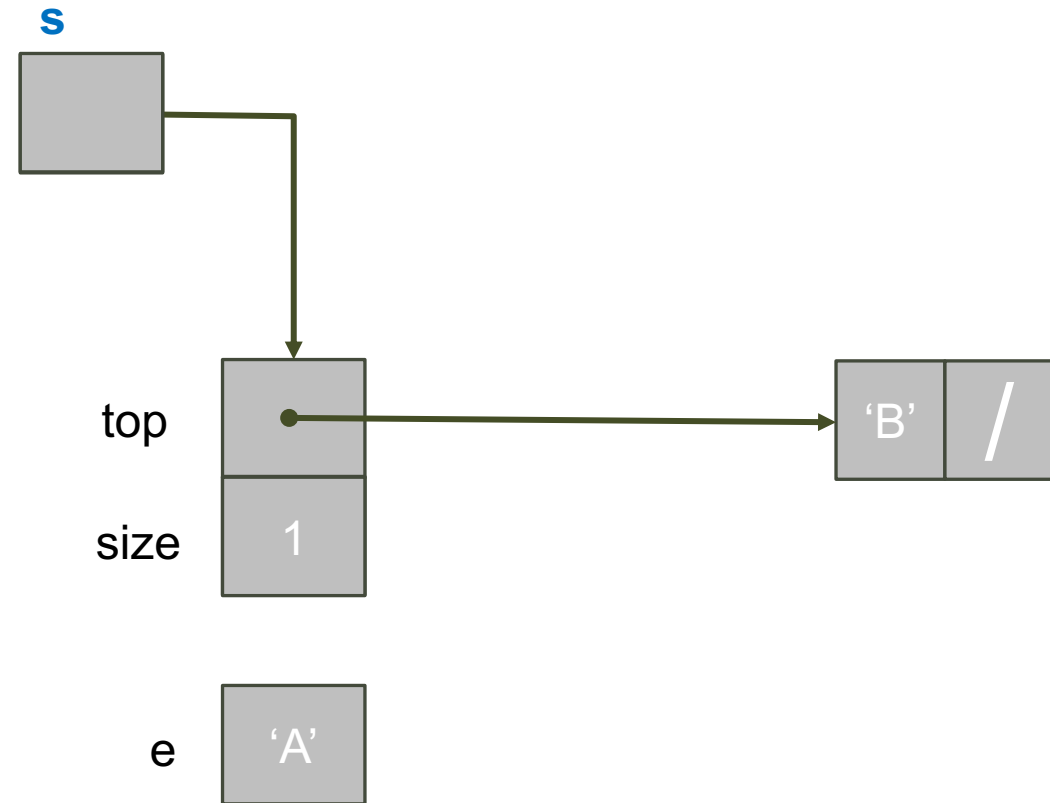
```
char pop(CharStackADT s) {  
    // Lista vuota?  
    if(isEmptyStack(s)){  
        exit(EXIT_FAILURE);  
    }  
    // Salvo dato in testa  
    char e = s->top->data;  
    // Cancella nodo di testa  
    ListNodePtr ptr = s->top;  
    s->top = ptr->next;  
    free(ptr);  
    // Decrementa dimensione stack  
    s->size--;  
}
```



Pila di char con Liste: pop ()

- Toglie e restituisce l'elemento in cima alla pila

```
char pop(CharStackADT s) {  
    // Lista vuota?  
    if(isEmptyStack(s)){  
        exit(EXIT_FAILURE);  
    }  
    // Salvo dato in testa  
    char e = s->top->data;  
    // Cancella nodo di testa  
    ListNodePtr ptr = s->top;  
    s->top = ptr->next;  
    free(ptr);  
    // Decrementa dimensione stack  
    s->size--;  
    // Restituisce dato  
    return e;  
}
```



Pila di char con Liste:

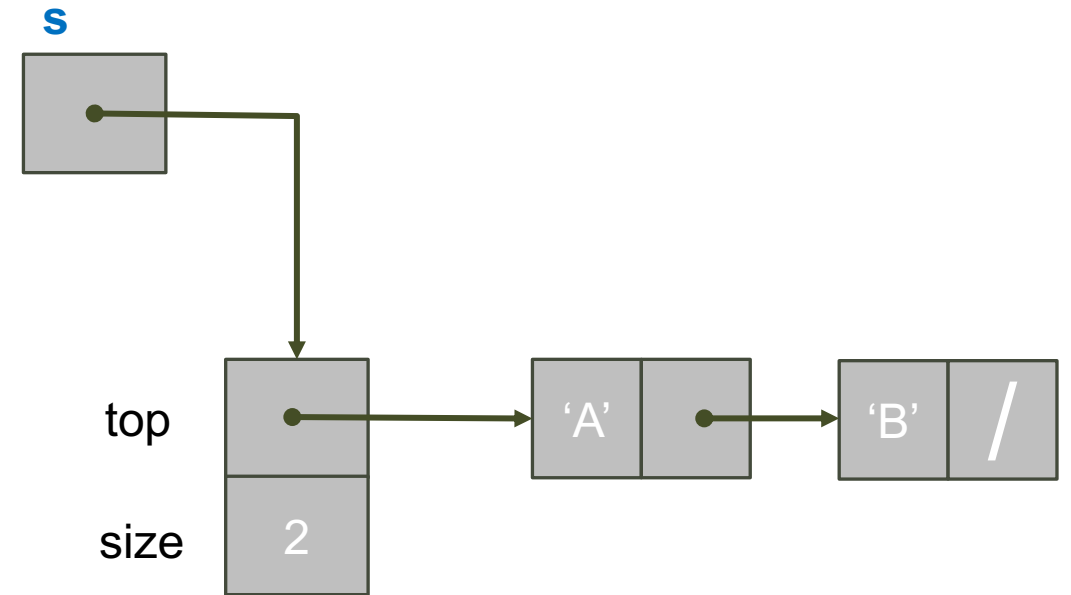
`isEmptyStack()` e `stackSize()`

- Controlla se stack è vuoto

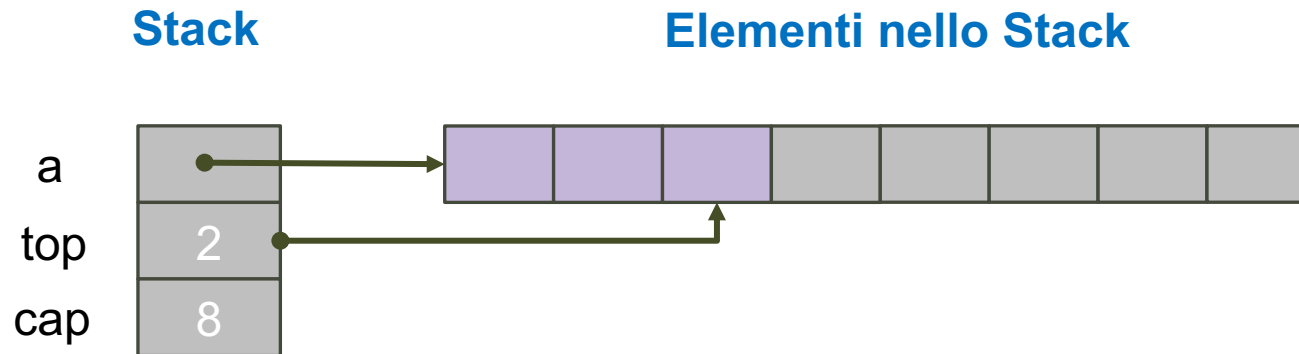
```
_Bool isEmptyStack(const CharStackADT s) {  
    return (s->top == NULL);  
}
```

- Restituisce dimensione stack

```
int stackSize(const CharStackADT s) {  
    return s->size;  
}
```



Pila di char con Array Dinamico



- Implementazione con array dinamico:
 - Elementi salvati nell'array
- Serve una variabile (top) che indichi fino a dove l'array è pieno
 - Ogni push/pop deve aggiornare l'indice top
- Se l'array è pieno (o troppo vuoto)
 - Ridimensioniamo array

Stack

```
struct charStack {  
    char* a; /* memorizza gli elementi  
              della pila */  
    int cap; /* dimensione  
              dell'array. */  
    int top; /* posizione dell'elemento  
              in cima alla pila */  
};
```

Pila di char con Array: mkStack ()

- Crea uno stack
 - Con array di dimensione INITIAL_CAPACITY e 0 elementi

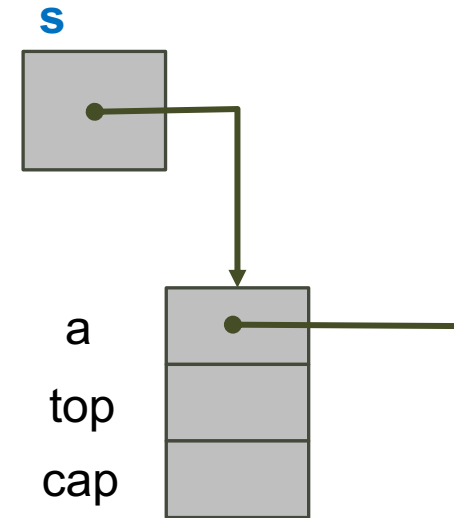
```
CharStackADT mkStack() {
```

```
}
```

Pila di char con Array: mkStack ()

- Crea uno stack
 - Con array di dimensione INITIAL_CAPACITY e 0 elementi

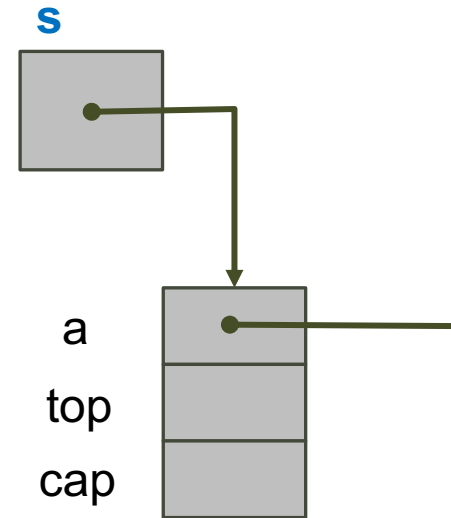
```
CharStackADT mkStack() {  
    // Alloca stack  
    CharStackADT s = malloc(sizeof(*s));  
  
}
```



Pila di char con Array: mkStack ()

- Crea uno stack
 - Con array di dimensione INITIAL_CAPACITY e 0 elementi

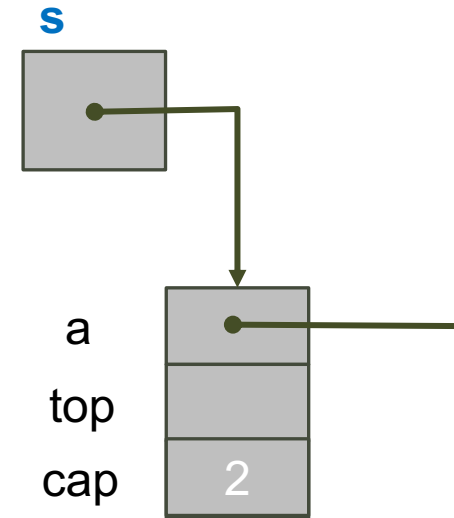
```
CharStackADT mkStack() {  
    // Alloca stack  
    CharStackADT s = malloc(sizeof(*s));  
    if(s == NULL) { // malloc fallita?  
        return NULL;  
    }  
}
```



Pila di char con Array: mkStack ()

- Crea uno stack
 - Con array di dimensione INITIAL_CAPACITY e 0 elementi

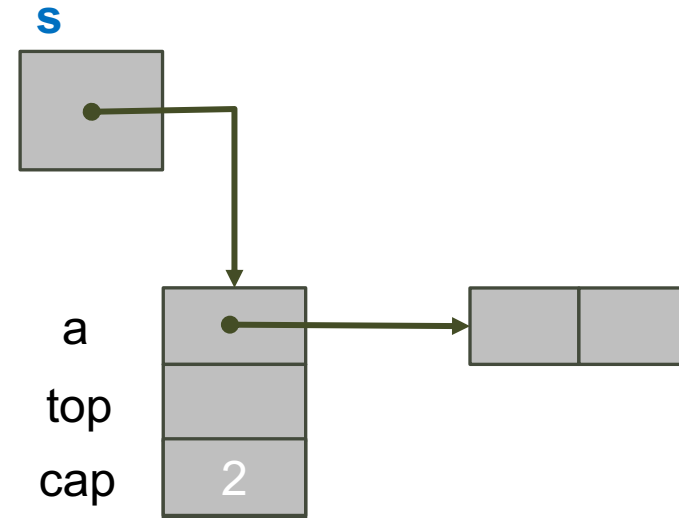
```
CharStackADT mkStack() {  
    // Alloca stack  
    CharStackADT s = malloc(sizeof(*s));  
    if(s == NULL) { // malloc fallita?  
        return NULL;  
    }  
    // Alloca array di dimensione INITIAL_CAPACITY  
    s->cap = INITIAL_CAPACITY;  
}
```



Pila di char con Array: mkStack ()

- Crea uno stack
 - Con array di dimensione INITIAL_CAPACITY e 0 elementi

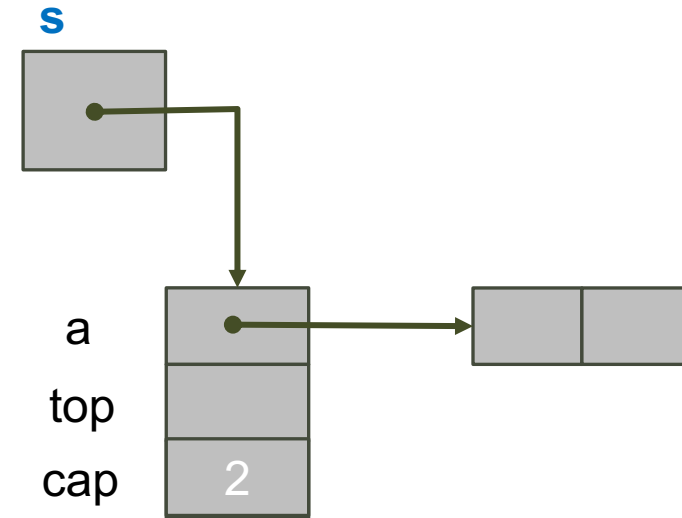
```
CharStackADT mkStack() {
    // Alloca stack
    CharStackADT s = malloc(sizeof(*s));
    if(s == NULL) { // malloc fallita?
        return NULL;
    }
    // Alloca array di dimensione INITIAL_CAPACITY
    s->cap = INITIAL_CAPACITY;
    s->a = malloc(s->cap * sizeof(*(s->a)));
}
```



Pila di char con Array: mkStack ()

- Crea uno stack
 - Con array di dimensione INITIAL_CAPACITY e 0 elementi

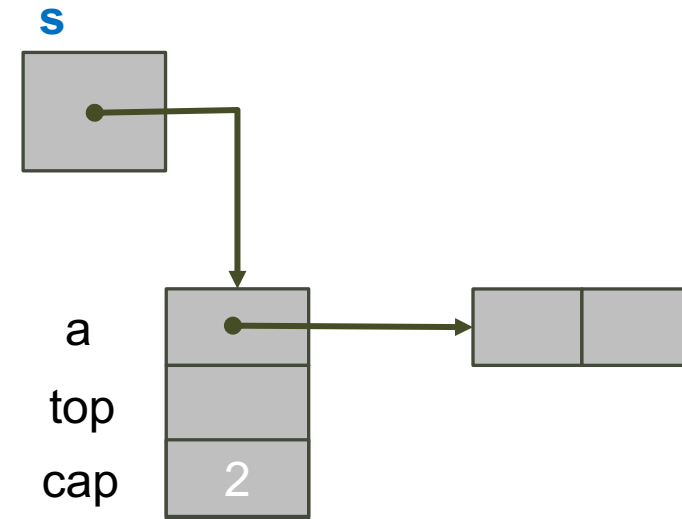
```
CharStackADT mkStack() {  
    // Alloca stack  
    CharStackADT s = malloc(sizeof(*s));  
    if(s == NULL) { // malloc fallita?  
        return NULL;  
    }  
    // Alloca array di dimensione INITIAL_CAPACITY  
    s->cap = INITIAL_CAPACITY;  
    s->a = malloc(s->cap * sizeof(*(s->a)));  
    if(s->a == NULL) { // malloc fallita?  
  
    }  
  
}
```



Pila di char con Array: mkStack ()

- Crea uno stack
 - Con array di dimensione INITIAL_CAPACITY e 0 elementi

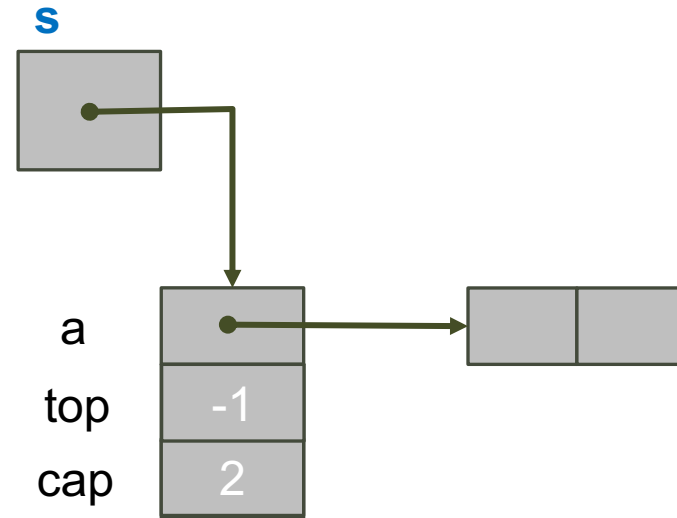
```
CharStackADT mkStack() {  
    // Alloca stack  
    CharStackADT s = malloc(sizeof(*s));  
    if(s == NULL) { // malloc fallita?  
        return NULL;  
    }  
    // Alloca array di dimensione INITIAL_CAPACITY  
    s->cap = INITIAL_CAPACITY;  
    s->a = malloc(s->cap * sizeof(*(s->a)));  
    if(s->a == NULL) { // malloc fallita?  
        // libera stack: o tutto o nulla  
        free(s); return NULL;  
    }  
}
```



Pila di char con Array: mkStack ()

- Crea uno stack
 - Con array di dimensione INITIAL_CAPACITY e 0 elementi

```
CharStackADT mkStack() {  
    // Alloca stack  
    CharStackADT s = malloc(sizeof(*s));  
    if(s == NULL) { // malloc fallita?  
        return NULL;  
    }  
    // Alloca array di dimensione INITIAL_CAPACITY  
    s->cap = INITIAL_CAPACITY;  
    s->a = malloc(s->cap * sizeof(*(s->a)));  
    if(s->a == NULL) { // malloc fallita?  
        // libera stack: o tutto o nulla  
        free(s); return NULL;  
    }  
    s->top = -1; // stack vuoto  
    return s;  
}
```



Pila di char con Array: dsStack ()

- Distrugge uno stack
 - Libera la memoria dell'array e dello stack



```
void dsStack(CharStackADT *ps) {  
    // Libera la memoria dell'array  
    free ((*ps)->a);  
  
}
```

Pila di char con Array: dsStack ()

- Distrugge uno stack
 - Libera la memoria dell'array e dello stack



```
void dsStack(CharStackADT *ps) {  
    // Libera la memoria dell'array  
    free ((*ps)->a);  
  
}
```

Pila di char con Array: dsStack ()

- Distrugge uno stack
 - Libera la memoria dell'array e dello stack



```
void dsStack(CharStackADT *ps) {  
    // Libera la memoria dell'array  
    free((*ps)->a);  
    // Libera la memoria dello stack  
    free(*ps);  
}
```

Pila di char con Array: dsStack ()

- Distrugge uno stack
 - Libera la memoria dell'array e dello stack



```
void dsStack(CharStackADT *ps) {  
    // Libera la memoria dell'array  
    free((*ps)->a);  
    // Libera la memoria dello stack  
    free(*ps);  
  
}
```


Pila di char con Array: dsStack ()

- Distrugge uno stack
 - Libera la memoria dell'array e dello stack

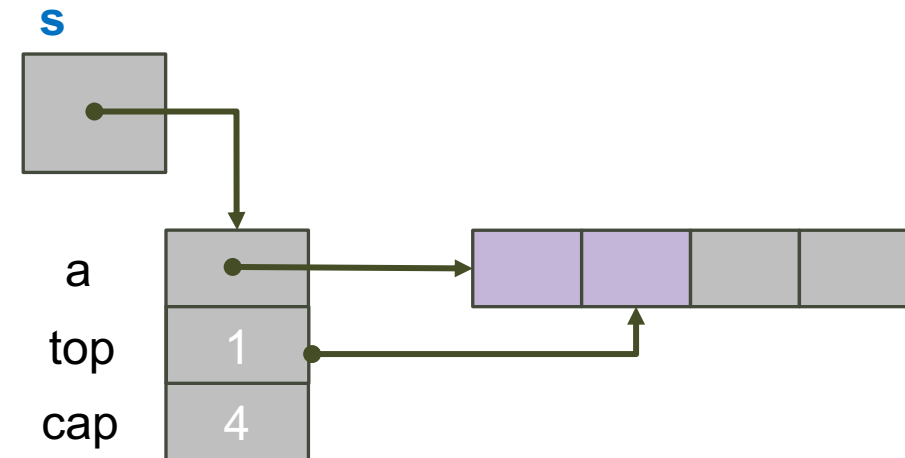
ps



```
void dsStack(CharStackADT *ps) {  
    // Libera la memoria dell'array  
    free((*ps)->a);  
    // Libera la memoria dello stack  
    free(*ps);  
    // Segna ps come NULL  
    *ps = NULL;  
}
```

Pila di char con Array: push () I

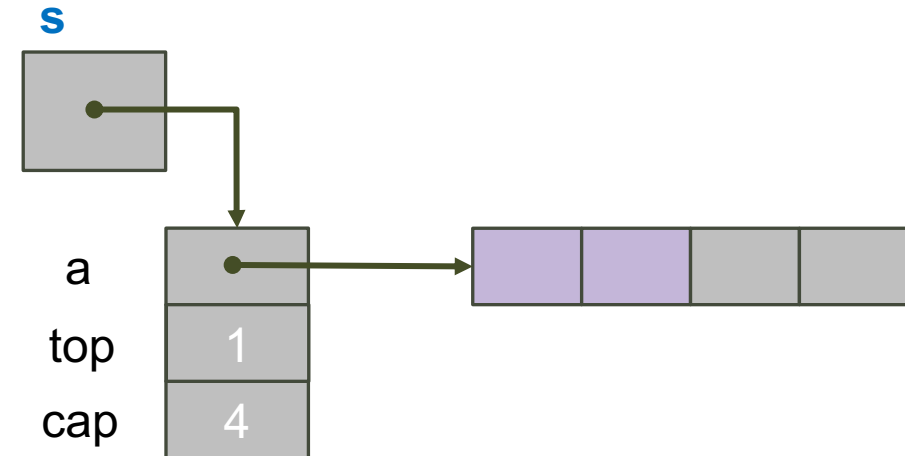
- Aggiunge elemento in posizione dopo top



```
_Bool push(CharStackADT s, char e) {  
    ...  
}
```

Pila di char con Array: push () I

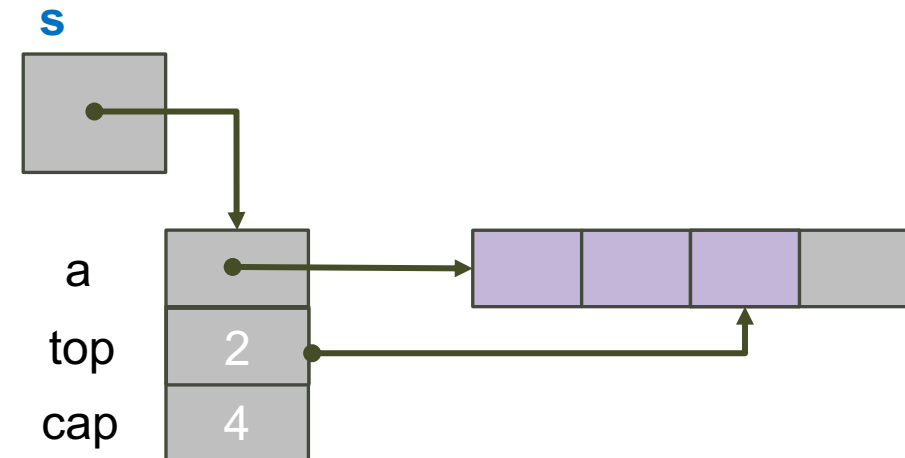
- Aggiunge elemento in posizione dopo `top`



```
_Bool push(CharStackADT s, char e) {  
    ...  
    // Inseriamo l'elemento nella posizione dopo s->top  
  
}
```

Pila di char con Array: push () I

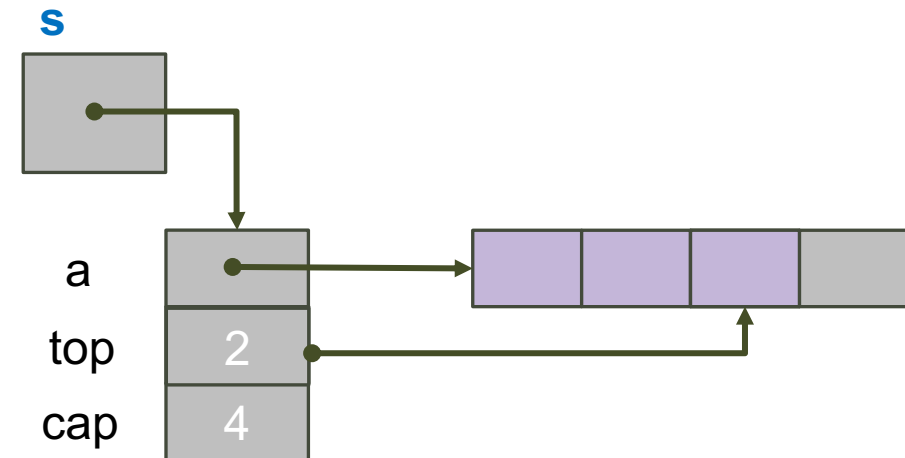
- Aggiunge elemento in posizione dopo `top`



```
_Bool push(CharStackADT s, char e) {
    ...
    // Inseriamo l'elemento nella posizione dopo s->top
    s->a[++(s->top)] = e;
    return 1;
}
```

Pila di char con Array: push () I

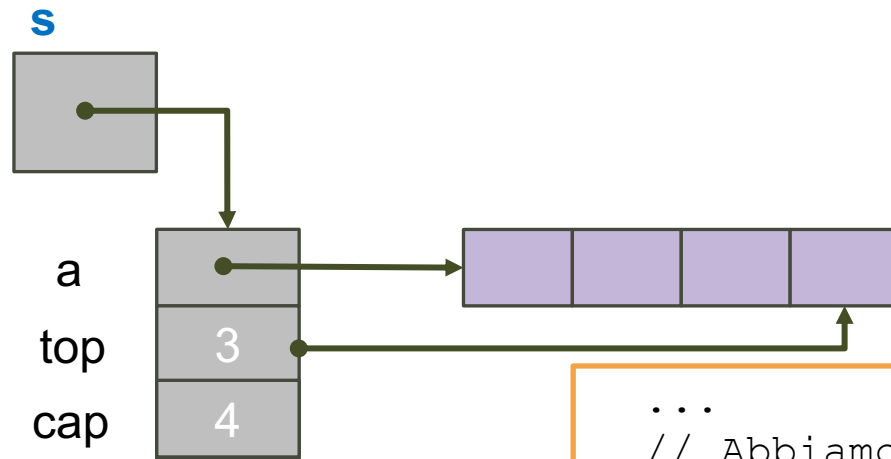
- Aggiunge elemento in posizione dopo top



```
_Bool push(CharStackADT s, char e) {
    ...
    // Inseriamo l'elemento nella posizione dopo s->top
    s->a[++(s->top)] = e;
    return 1;
}
```

Problema:
se cresce oltre a cap?

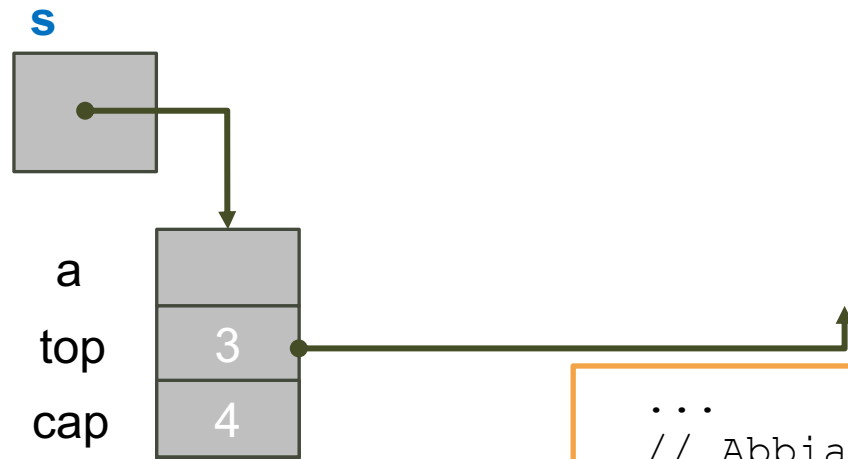
Pila di char con Array: push () II



...
// Abbiamo finito lo spazio?

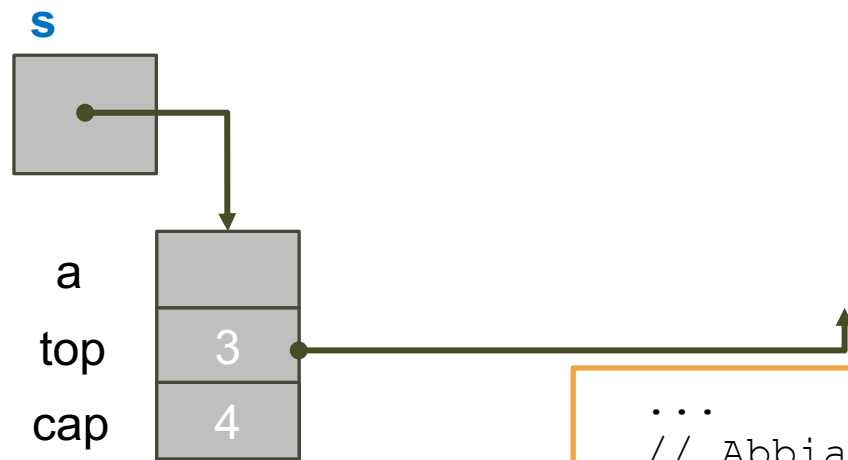
...

Pila di char con Array: push () II



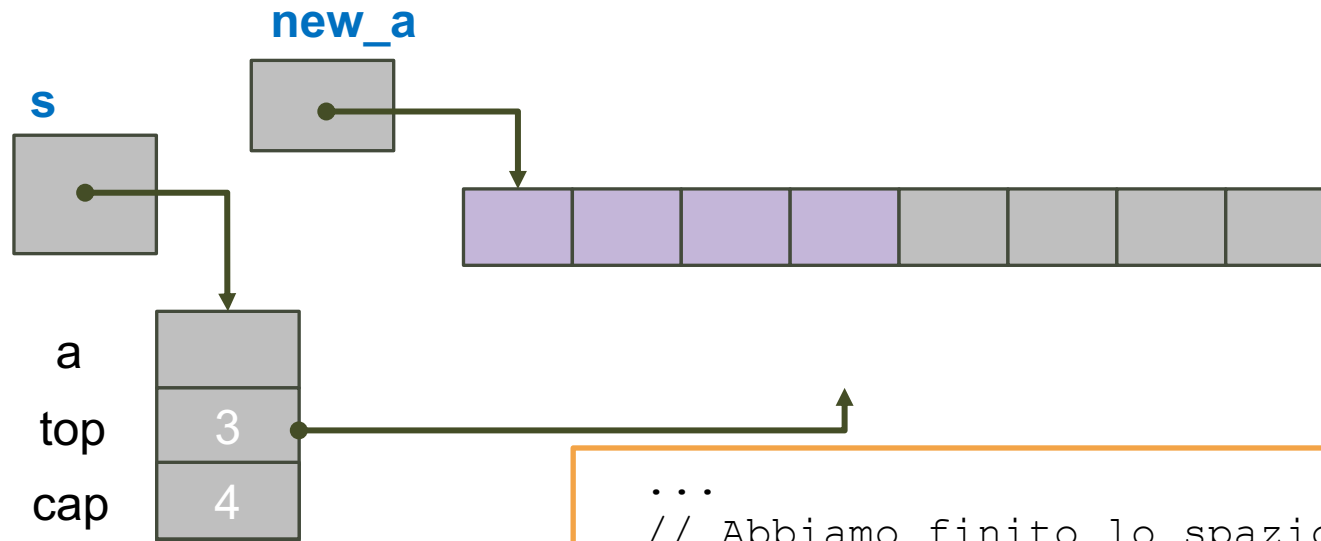
```
...  
// Abbiamo finito lo spazio?  
if (s->top == s->cap-1) {  
  
}  
...
```

Pila di char con Array: push () II



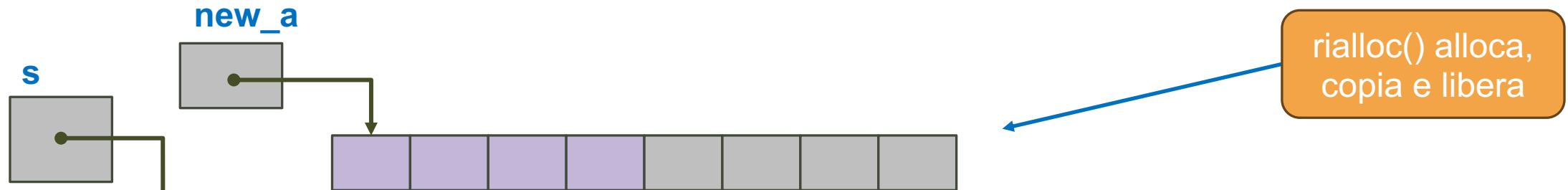
```
...  
// Abbiamo finito lo spazio?  
if (s->top == s->cap-1) {  
    // Proviamo a riallocare l'array con il doppio della capacità  
    char* new_a = realloc(s->a, s->cap * 2 * sizeof(*(s->a)));  
  
}  
...
```


Pila di char con Array: push () II



```
...  
// Abbiamo finito lo spazio?  
if (s->top == s->cap-1) {  
    // Proviamo a riallocare l'array con il doppio della capacità  
    char* new_a = realloc(s->a, s->cap * 2 * sizeof(*(s->a)));  
  
}  
...
```

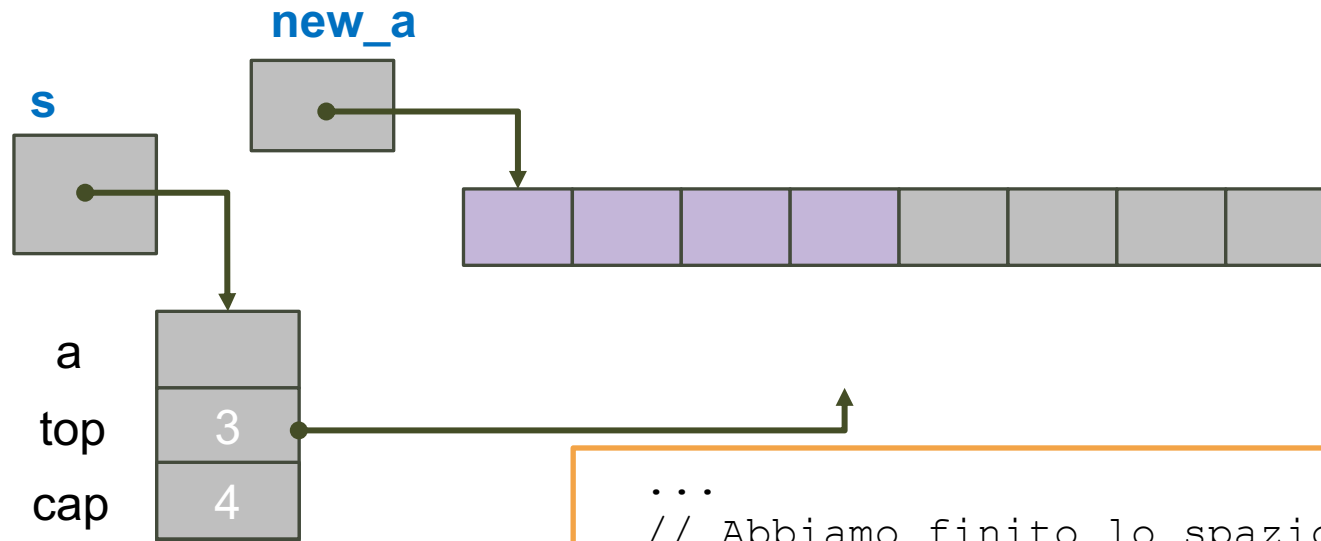
Pila di char con Array: push () II



realloc() alloca,
copia e libera

```
...  
// Abbiamo finito lo spazio?  
if (s->top == s->cap-1) {  
    // Proviamo a riallocare l'array con il doppio della capacità  
    char* new_a = realloc(s->a, s->cap * 2 * sizeof(*(s->a)));  
  
}  
...
```

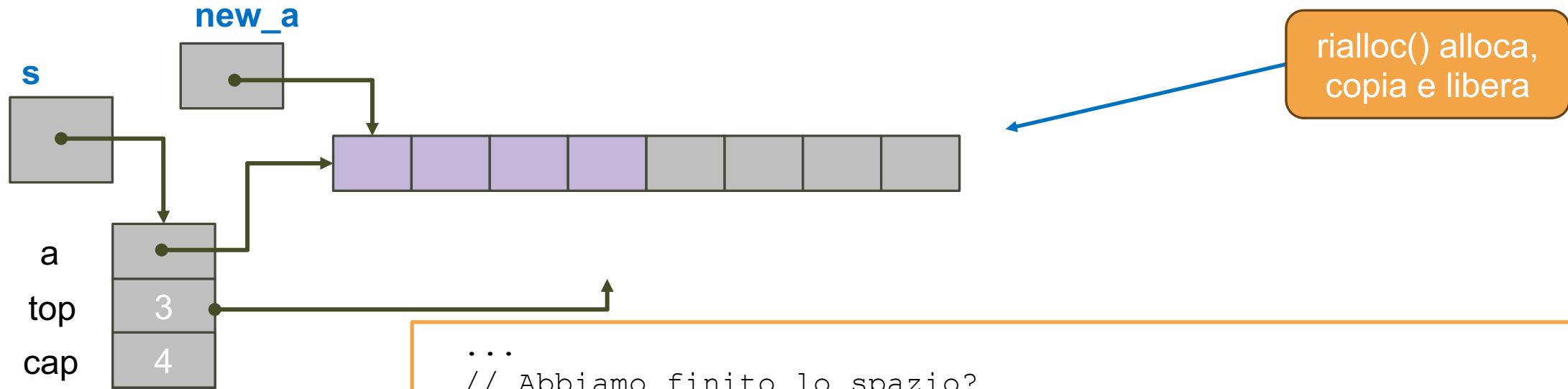
Pila di char con Array: push () II



realloc() alloca,
copia e libera

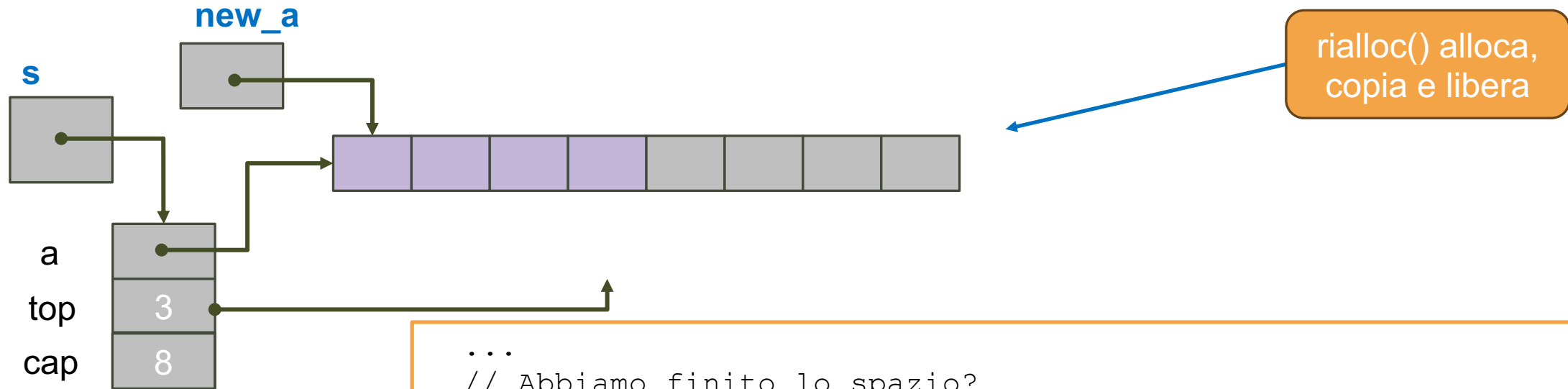
```
...  
// Abbiamo finito lo spazio?  
if (s->top == s->cap-1) {  
    // Proviamo a riallocare l'array con il doppio della capacità  
    char* new_a = realloc(s->a, s->cap * 2 * sizeof(*(s->a)));  
    if(new_a == NULL) { return 0;} // realloc fallita?  
  
}  
...
```

Pila di char con Array: push () II



```
...  
// Abbiamo finito lo spazio?  
if (s->top == s->cap-1) {  
    // Proviamo a riallocare l'array con il doppio della capacità  
    char* new_a = realloc(s->a, s->cap * 2 * sizeof(*(s->a)));  
    if(new_a == NULL) { return 0;} // realloc fallita?  
    s->a = new_a; // Il new_a sostituisce quello precedente  
}  
...
```

Pila di char con Array: push () II

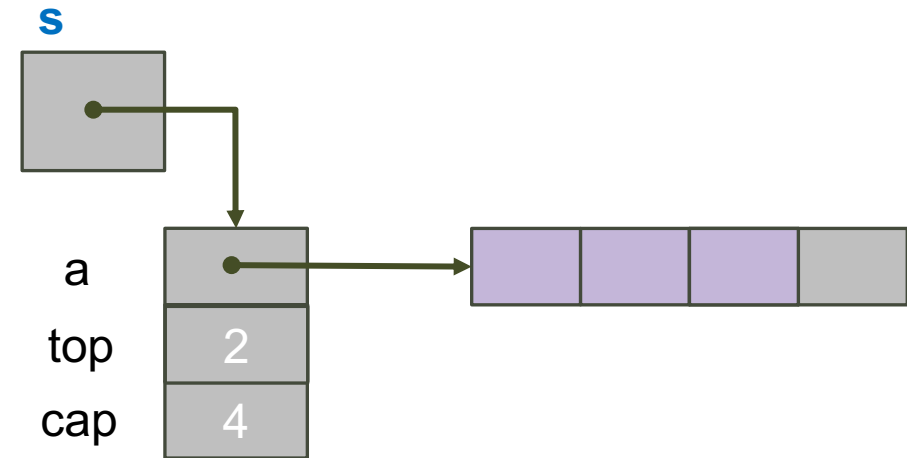


```
...
// Abbiamo finito lo spazio?
if (s->top == s->cap-1) {
    // Proviamo a riallocare l'array con il doppio della capacità
    char* new_a = realloc(s->a, s->cap * 2 * sizeof(*(s->a)));
    if(new_a == NULL) { return 0;} // realloc fallita?
    s->a = new_a; // Il new_a sostituisce quello precedente
    s->cap *= 2;
}
...
```

Pila di char con Array: pop () I

- Toglie e restituisce l'elemento puntato da `top`

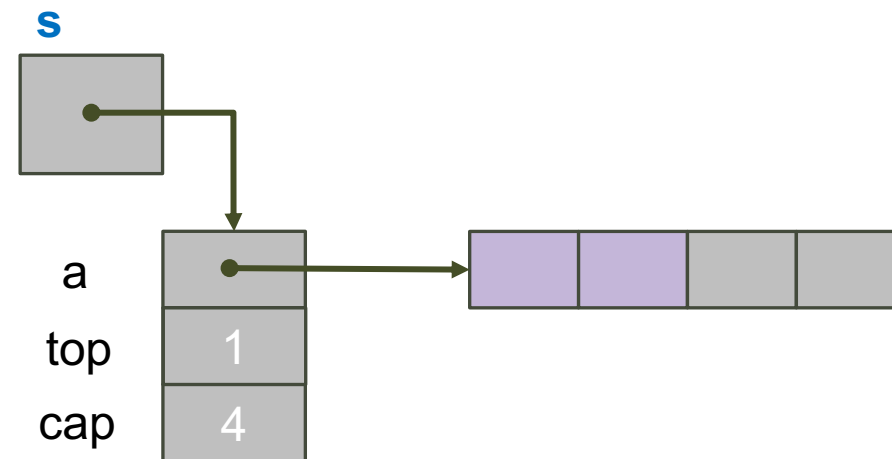
```
char pop(CharStackADT s) {  
  
    ...  
    return e;  
}
```



Pila di char con Array: pop () I

- Toglie e restituisce l'elemento puntato da `top`

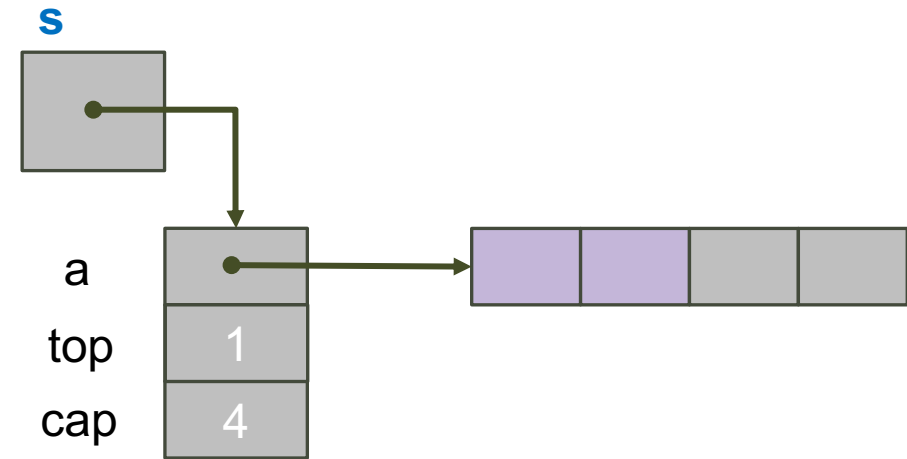
```
char pop(CharStackADT s) {  
    // Pila vuota?  
    if(isEmptyStack(s)){  
        exit(EXIT_FAILURE);  
    }  
  
    ...  
    return e;  
}
```



Pila di char con Array: pop () I

- Toglie e restituisce l'elemento puntato da `top`

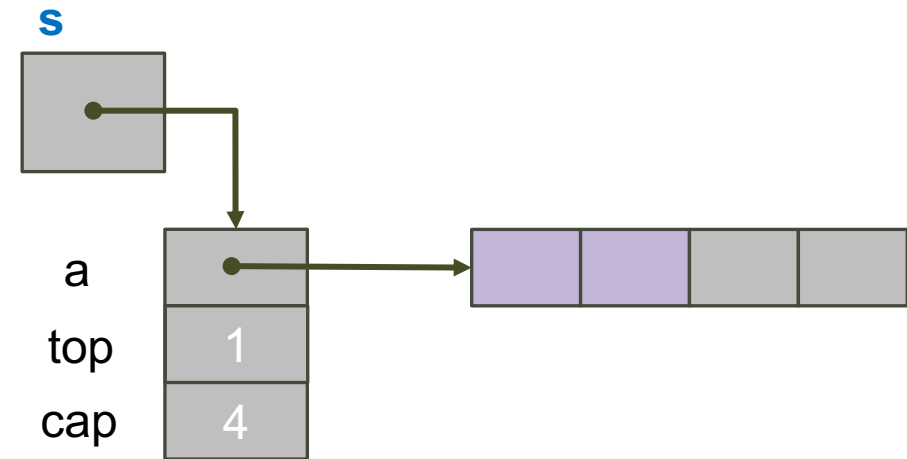
```
char pop(CharStackADT s) {  
    // Pila vuota?  
    if(isEmptyStack(s)){  
        exit(EXIT_FAILURE);  
    }  
    // Restituisco l'elemento in posizione top  
    char e = s->a[(s->top)--];  
    ...  
    return e;  
}
```



Pila di char con Array: pop () I

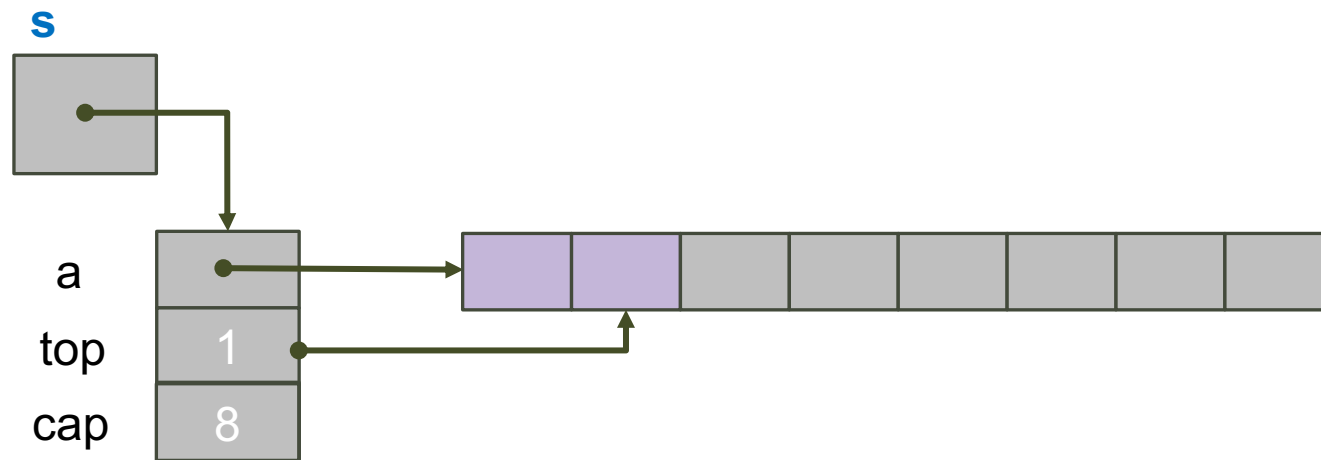
- Toglie e restituisce l'elemento puntato da `top`

```
char pop(CharStackADT s) {
    // Pila vuota?
    if(isEmptyStack(s)){
        exit(EXIT_FAILURE);
    }
    // Restituisco l'elemento in posizione top
    char e = s->a[(s->top)--];
    ...
    return e;
}
```



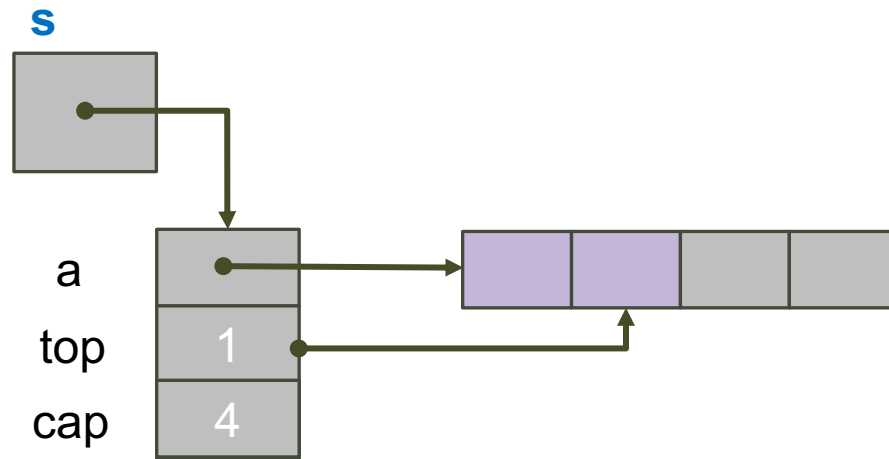
Finito?

Pila di char con Array: pop () II



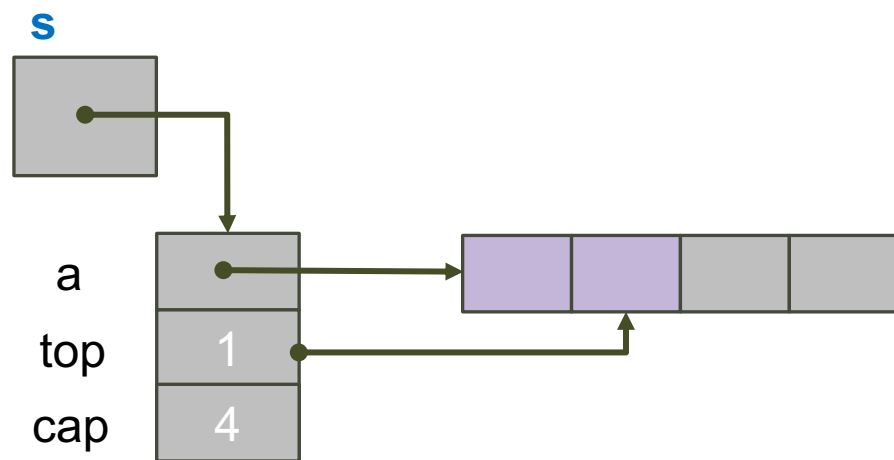
...
// Possiamo ridurre l'array?

Pila di char con Array: pop () II



```
...  
// Possiamo ridurre l'array?  
if ((s->cap > INITIAL_CAPACITY) && (s->top == s->cap/4)) {  
  
}
```

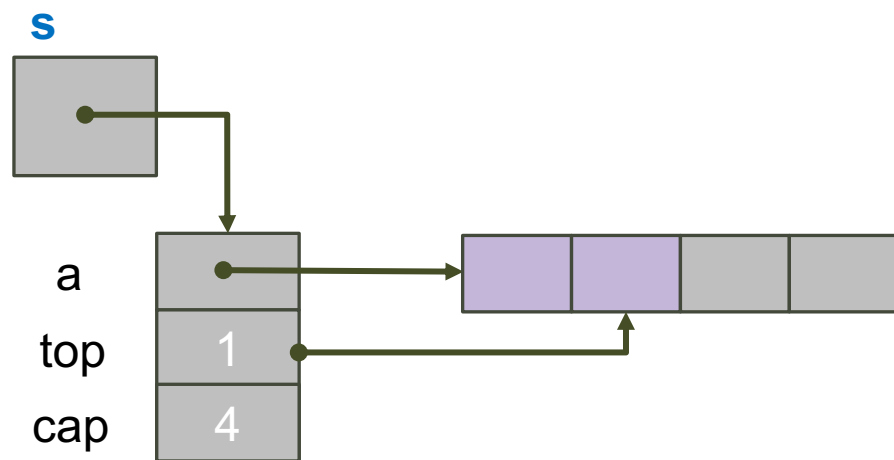
Pila di char con Array: pop () II



Perché cap/4?

```
...  
// Possiamo ridurre l'array?  
if ((s->cap > INITIAL_CAPACITY) && (s->top == s->cap/4)) {  
  
}
```

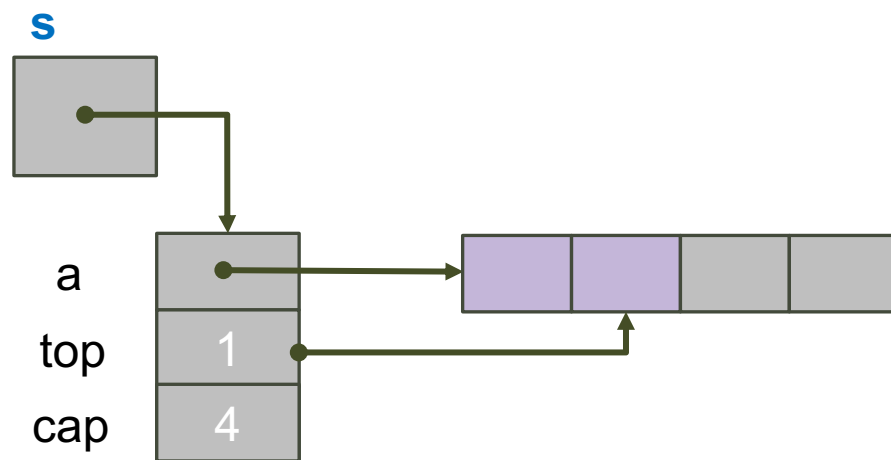
Pila di char con Array: pop () II



Perché cap/4?

```
...  
// Possiamo ridurre l'array?  
if ((s->cap > INITIAL_CAPACITY) && (s->top == s->cap/4)) {  
    s->cap /= 2;  
}
```

Pila di char con Array: pop () II



Perché cap/4?

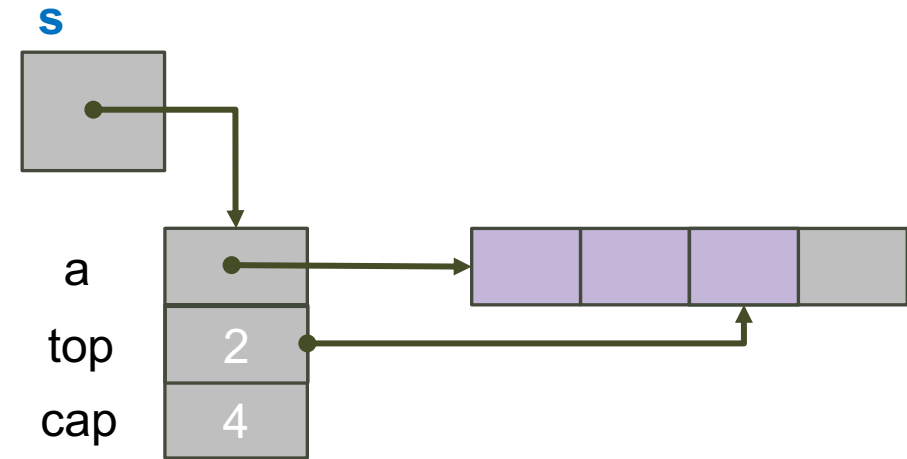
```
...  
// Possiamo ridurre l'array?  
if ((s->cap > INITIAL_CAPACITY) && (s->top == s->cap/4)) {  
    s->cap /= 2;  
    s->a = realloc(s->a, s->cap * sizeof(*(s->a)));  
}
```

Pila di char con Array:

`isEmptyStack()` e `stackSize()`

- Controlla se stack è vuoto

```
_Bool isEmptyStack(const CharStackADT s) {
}
```

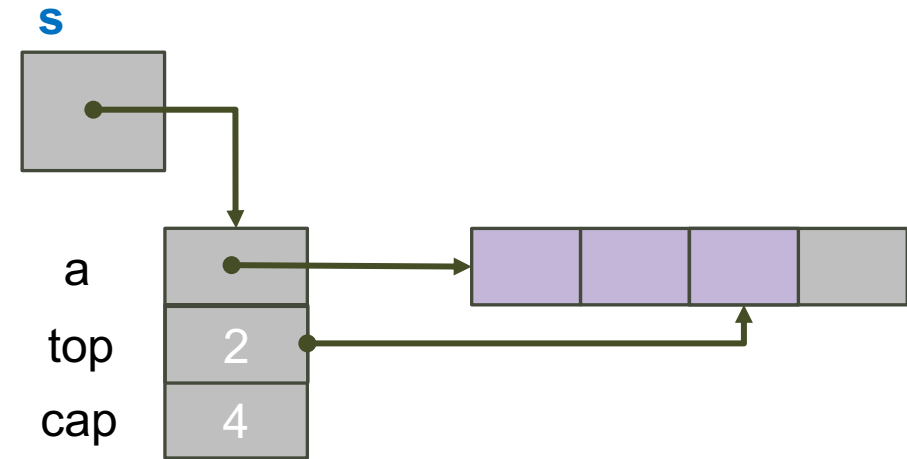


Pila di char con Array:

`isEmptyStack()` e `stackSize()`

- Controlla se stack è vuoto

```
_Bool isEmptyStack(const CharStackADT s) {
    return (s->top < 0);
}
```



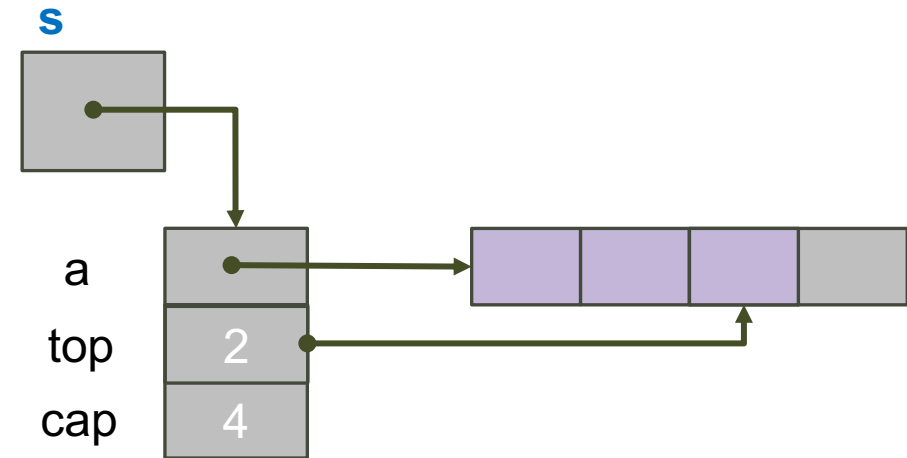
Pila di char con Array:

`isEmptyStack()` e `stackSize()`

- Controlla se stack è vuoto

```
_Bool isEmptyStack(const CharStackADT s) {  
    return (s->top < 0);  
}
```

- Restituisce dimensione stack



Pila di char con Array:

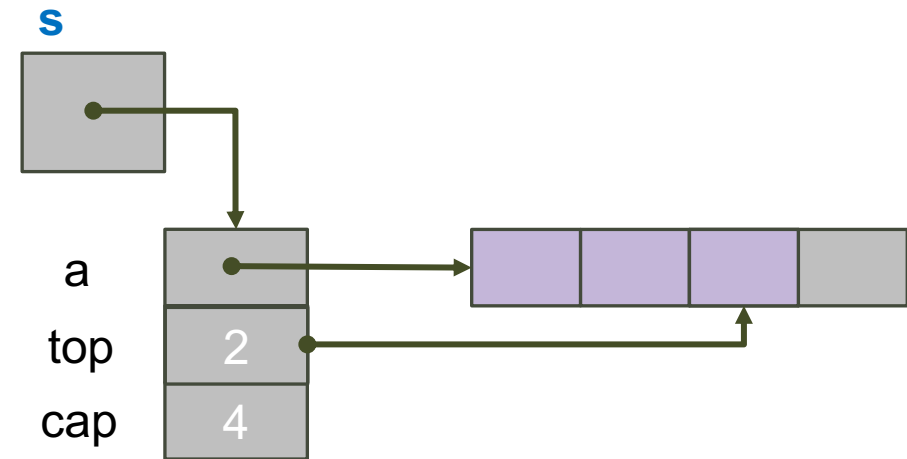
`isEmptyStack()` e `stackSize()`

- Controlla se stack è vuoto

```
_Bool isEmptyStack(const CharStackADT s) {
    return (s->top < 0);
}
```

- Restituisce dimensione stack

```
int stackSize(const CharStackADT s) {
}
```



Pila di char con Array:

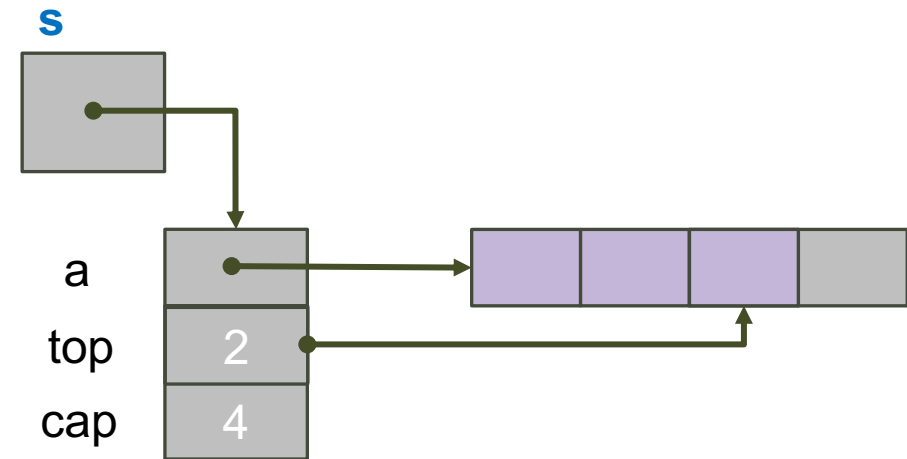
`isEmptyStack()` e `stackSize()`

- Controlla se stack è vuoto

```
_Bool isEmptyStack(const CharStackADT s) {
    return (s->top < 0);
}
```

- Restituisce dimensione stack

```
int stackSize(const CharStackADT s) {
    return (s->top + 1);
}
```



Principali Applicazioni degli Stack

- Struttura dati molto semplice ma molto “potente”:
 - **Esecuzione delle chiamate a funzione** (già studiato)
 - **Inversione di una stringa**: sequenza di push delle singole lettere su uno stack e (terminata la stringa) sequenza di pop. L'ordine inverso è ottenuto grazie alla natura LIFO dello stack
 - **Compiler**: calcolo del valore di espressioni (es. $2 + 4 / 5 * (7 - 9)$) convertendole in forma prefissa o postfissa
 - **Browser**: il pulsante back di un browser salva tutti gli URL visitati in uno stack. Ogni visita di una nuova pagina è aggiunta con push, ogni click su back è un pop che porta alla pagina visitata in precedenza
 - **Sistemi operativi**: implementazione delle politiche di sostituzione delle pagine “least recently used”